Rewriting the Universe

High-Performance Hypergraph Evolution for the Wolfram Physics Project

Richard Assar
Wolfram Institute
rassar@wolframinstitute.org

August 29, 2026

Abstract

The Wolfram Physics Project models fundamental physics through hypergraph rewriting systems, where the evolution of simple rules generates complex causal and branchial structures. However, exhaustive multiway evolution is computationally intensive, with naive implementations exhibiting exponential time complexity in the number of evolution steps. We present a high-performance implementation whose three principal components are: (1) exact graph canonicalization by McKay-style individualization–refinement (IR), which under the exact mode runs on every state and whose hash serves directly as the deduplication key. We show that interposing a cheaper Weisfeiler–Leman filter cannot reduce the number of IR invocations by more than the ratio of canonical classes to raw states, a ratio that decreases with evolution depth; (2) lock-free concurrent data structures enabling parallel evolution across many CPU cores; and (3) a GPU evolution backend. Canonicalization is validated for exactness against a brute-force isomorphism oracle and cross-checked against the `Wolfram/Multicomputation` paclet's `MultiwaySystem`, so state and event deduplication are exact rather than heuristic.

The engine is compared against two implementations of the same definition, run on the same rule and initial condition, and agreeing with the engine on state and causal-edge counts at every depth. The first is the `Wolfram/Multicomputation` paclet's `MultiwaySystem`, the implementation a user would otherwise run; the second is this project's own brute-force reference implementation, which returns data rather than graph objects and so measures the definition without the cost of constructing them. Against the first, the engine is faster by a factor of 2,770 at depth 4, rising to 30,627 at depth 7. The system is available as a standalone C++/CUDA library and as a Wolfram Language paclet.

Every quantitative table and figure in Section 8 is generated from the tool that produces it, and each carries the commit, the machine and the tool in its own provenance line. Wall-clock figures are medians over repeated runs.

Keywords: hypergraph rewriting, graph canonicalization, multiway systems, GPU computing, Wolfram Physics Project

1 Introduction

The Wolfram Physics Project [1] proposes that the fundamental structure of the universe emerges from the repeated application of simple rewriting rules to hypergraphs. This approach generates three derived structures: causal graphs, recording dependencies between rewriting events; branchial graphs, connecting states reachable at the same evolution step; and multiway evolution graphs, representing the set of all reachable histories.

Exhaustive multiway evolution is computationally demanding. The number of reachable states grows exponentially with the number of evolution steps, and each step requires pattern

matching against the current hypergraphs, canonicalization to identify isomorphic duplicates, and event recording to construct the causal and branchial structures. The existing Wolfram Language implementation is mathematically complete, but slow enough that its running time rather than the definition sets what can be explored. Section 8 measures it at 3.7 s to reach the six canonical states of depth five on the corpus's `wolfram-2to4` rule, against 0.3 ms for the engine on the same evolution, and the gap widens with depth.

This paper presents a high-performance implementation addressing these challenges through:

1. **Exact canonicalization:** We use McKay-style individualization–refinement (IR) [2] as the canonicalizer, and under the exact mode it runs on every state: its hash is the deduplication key, so two states are identified if and only if they are isomorphic. A Weisfeiler–Leman signature [3] serves the approximate modes, which exist for callers that ask for a coarser identity. Proposition 3.3 shows why it is not placed in front of IR: the saving available to such a filter is bounded above by the ratio of canonical classes to raw states, which falls toward zero as an evolution deepens.

2. **Lock-free concurrency:** The engine holds no lock. Its concurrent hash maps, segmented arrays, lock-free lists and work-stealing deques are non-blocking, and a worker with no work waits on an address rather than spinning.

3. **GPU acceleration:** A CUDA backend runs matching, rewriting and canonicalization in one kernel that stays resident across evolution steps, so no step returns to the host. It is faster than the host where the rule set yields enough independent work to fill the device, which Section 8 quantifies.

4. **Configurable equivalencing:** State and event equivalencing are independent axes. State canonicalization is selected as None, Automatic (fast content hash), or Full (exact IR); event canonicalization is composed from a small set of signature keys. There is no user-facing choice between competing canonicalization algorithms—the exact path is always IR, computed on every state.

5. **Exploration strategies:** Quotient exploration (Section 5.10) expands one representative per isomorphism class while retaining the full event set; transition-level sampling with per-rule weights (Section 5.12) admits runs at depths where exhaustive expansion is not available; and the causal graph's transitive reduction (Section 5.11) is maintained online rather than as a pass over the finished graph.

6. **A determinism contract:** The state set, event set and causal and branchial relations are a function of the inputs alone, independent of thread count and scheduling order (Section 5.8).

The remainder of this paper is organized as follows. Section 2 reviews hypergraph rewriting and the Wolfram Physics framework. Section 3 presents our canonicalization algorithms. Section 4 describes the pattern matching system. Section 5 details the system architecture. Section 6 covers GPU acceleration. Section 8 presents experimental results. Section 9 concludes with future directions.

2 Background and Related Work

This section fixes the objects the rest of the paper measures and states which of their properties are the expensive ones. Hypergraph rewriting supplies the rule and the match; multiway evolution supplies the branching structure that makes a state space rather than a trajectory, together with the causal and branchial relations derived from it. The cost of the whole construction

concentrates in one place: to know whether two states are the same state, the system must decide hypergraph isomorphism, and it must decide it once per produced state rather than once per query. We close with the systems and libraries this work is measured against.

2.1 Hypergraph Rewriting Systems

A *hypergraph* $\mathcal{H} = (V, E)$ consists of a vertex set V and a set of hyperedges E, where each hyperedge $e \in E$ is an ordered tuple of vertices $(v_1, v_2, \ldots, v_k)$ with $v_i \in V$. Unlike standard graphs where edges connect exactly two vertices, hyperedges may connect any number of vertices with arbitrary arity.

Definition 2.1 (Rewriting Rule)**.** *A rewriting rule $\mathcal{R} = (L, R)$ consists of a left-hand side pattern L and right-hand side replacement R, both hypergraphs. A* match *of L in $\mathcal{H}$ is an injective mapping $\phi : V_L \to V_{\mathcal{H}}$ such that the image of each hyperedge in L appears in $\mathcal{H}$.*

This is the single-pushout form of rewriting: the rule is applied by deleting the matched image and gluing in the replacement, without the additional gluing conditions that the double-pushout construction imposes [4–6]. Applying rule $\mathcal{R}$ at match ϕ produces a new hypergraph $\mathcal{H}'$ by:

1. Removing all hyperedges in $\phi(L)$ from $\mathcal{H}$
2. Adding hyperedges from R, extending ϕ to map new vertices in R to fresh vertices

2.2 Multiway Evolution

Multiway evolution explores all possible rule applications simultaneously, generating a directed acyclic graph of states connected by events. Figure 1 shows two steps of a small rule in full—the raw states, the canonical states they collapse to, the events between them and the causal edges—and is the smallest complete instance of everything the rest of the paper computes.

Definition 2.2 (Evolution Graph)**.** *Given initial state $\mathcal{S}_0$ and rule set $\{\mathcal{R}_i\}$, the evolution graph $G = (\mathcal{S}, \mathcal{E})$ contains:*

- *States $\mathcal{S}$: all hypergraphs reachable from $\mathcal{S}_0$*
- *Events $\mathcal{E}$: transitions (s, s', r, ϕ) where state s rewrites to s' via rule r at match ϕ*

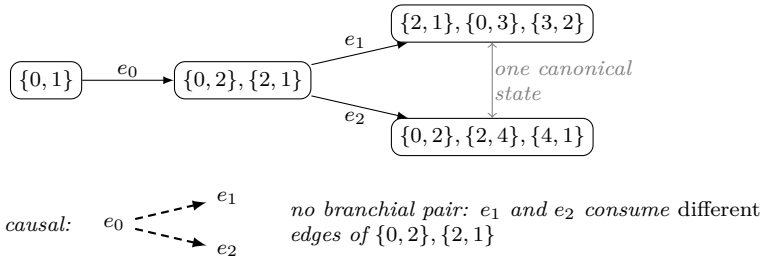

Figure 1: Two steps of $\{x, y\} \to \{x, z\}, \{z, y\}$ from a single edge, exactly as the engine reports it. Four raw states become three canonical ones: the two depth-2 results are isomorphic and are identified. Both causal edges descend from e_0, because each depth-2 event consumes one of the two edges e_0 produced. The branchial relation is empty here, and for a reason worth stating: e_1 and e_2 share an input state, which resembles a sibling pair, but branchial edges connect co-consumers of a *common* edge and these consume different ones. Every element is output of `canonical_causal_oracle --dump 2`.

The *causal graph* captures dependencies between events based on shared hyperedges. The *branchial graph* connects states reachable at the same evolution step, encoding branching structure analogous to quantum superposition [1].

2.3 The Graph Isomorphism Problem

State deduplication requires detecting when two hypergraphs are isomorphic—structurally identical up to vertex relabeling. Graph isomorphism is in NP and is neither known to be NP-complete nor known to be in P [7]; the best known bound is quasipolynomial [8].

For practical systems, we employ *graph hashing*: computing a canonical signature $h(\mathcal{H})$ such that isomorphic graphs have identical hashes. While perfect hashing (injective for non-isomorphic graphs) requires solving isomorphism exactly, approximate hashes suffice when combined with explicit isomorphism testing for collisions.

2.4 Related Work

Graph canonicalization: The nauty/Traces line of work [2] establishes individualization–refinement (IR) as the practical standard for exact graph canonicalization; it is exponential in the worst case but fast on typical instances, and it is the exact canonicalizer we adopt as our reference. Weisfeiler–Leman color refinement [3, 9] runs in polynomial time but provably fails to distinguish certain non-isomorphic graphs, so it cannot serve as an identity where exactness is required; we use it only for the modes that ask for a deliberately coarser identity, never in front of IR.

Hypergraph rewriting implementations: SetReplace [10] is the implementation most closely associated with the Wolfram Physics Project, and is the closest system to this work. Throughout this paper, *reference implementation* denotes this project's own brute-force evolver, named below, and never SetReplace. The two target different objects. SetReplace's multiway construction is a token–event graph over a single growing token pool: it records which rewrites occurred and what each consumed and produced, and does not canonicalize or deduplicate states during evolution, so two structurally identical states reached along different histories remain distinct vertices. The object this engine enumerates is the states graph under an exact isomorphism identity, which requires canonicalizing every state as it is produced (Section 3); canonicalization therefore dominates the cost, rather than matching. The two objects are complementary rather than competing: a token–event graph is cheaper and suits an analysis of history, while a deduplicated states graph suits an analysis of the reachable state space. We compare against two implementations in Section 8, both of which compute the same object we do: the `Wolfram/Multicomputation` paclet's `MultiwaySystem` [11], and this project's own reference implementation (`reference/MultiwayReference.wl`), a direct transcription of the definition used as the correctness oracle throughout this work. The two answer different questions. `MultiwaySystem` is the implementation a user would otherwise run, but every property it exposes is graph-valued, so its time necessarily includes building Wolfram `Graph` objects and no subtraction can remove that from the outside. The reference returns data, so its time is the closest Wolfram-side analogue of the engine's own native measurement. SetReplace computes a different object, so a wall-time comparison against it would compare two different computations.

Subgraph matching on hypergraphs: HGMatch [12] matches by hyperedge rather than vertex by vertex, seeding a join from one pattern hyperedge and extending it one hyperedge at a time, and partitions candidate hyperedges by a label-derived signature to restrict enumeration. HGMatch also organizes execution as a dataflow with a task scheduler performing fine-grained dynamic work stealing. Our matcher adopts all three. The join is organized as a seeding scan followed by expansions that each bind one further pattern edge (Section 4.3), and because hyperedges here carry no labels, the signature that partitions candidates is instead the vertex-repetition pattern of an edge (Definition 4.1), which plays the same filtering role for unlabelled data. Where an expansion has a variable already bound, candidates come from the vertex index rather than the signature bucket; on the device this replaced a global signature walk and reduced match-kernel time on one workload from 944 ms to 4 ms.

GPU graph algorithms: Recent work has explored GPU acceleration for subgraph match-

ing [13], graph mining [14], and general graph algorithms [15]. The warp-centric organization of our match kernel—one block per (state, rule) pair, threads striding over seed candidates and each running its own depth-first subtree—follows G2Miner [16]. Our kernel differs from these systems in remaining resident across evolution steps rather than being relaunched per step, because the workload is iterative: each step's states are the next step's inputs.

Dynamic graph algorithms: The causal graph's transitive reduction is maintained as edges arrive rather than recomputed, which places it in the dynamic graph-algorithms setting; Goranci et al. [17] give fully dynamic algorithms for transitive reduction under insertions and deletions, in $\mathcal{O}(m + n \log n)$ amortized time. Our setting admits insertions only, arriving in topological order, so it is a restriction of that problem rather than an instance of it, and we claim no result against it (Section 5.11).

Term rewriting: The theory of term rewriting [18] treats confluence and termination for tree-structured terms, and high-performance implementations [19] optimise similar pattern-matching operations. Hypergraph rewriting differs in that the objects are graph-structured, so matching is subgraph matching rather than tree matching, and states must be compared up to isomorphism rather than syntactic equality.

3 Graph Canonicalization

State deduplication determines how large the explored set becomes: without isomorphism detection, every path to a state contributes a separate vertex and the set grows exponentially with depth. Deduplication must be *exact*: two states are the same node if and only if their hypergraphs are isomorphic (vertex relabeling, within-edge order and multiplicity preserved). The exact mode satisfies this requirement using individualization–refinement (IR) alone: it is computed for every state, and its hash serves directly as the deduplication key. Weisfeiler–Leman (WL) is a separate, coarser signature, used by the modes that request a coarser notion of state identity.

3.1 Individualization–Refinement

The exact canonical form is computed by McKay-style individualization–refinement [2], the same family of algorithms underlying nauty and Traces. IR produces a canonical labeling—and, as a byproduct, a set of automorphism generators and vertex/edge orbits—such that two hypergraphs receive identical canonical forms if and only if they are isomorphic.

Definition 3.1 (Canonical Form)**.** *A canonicalization* Canon *maps each hypergraph to a labeled representative such that* $\textsc{Canon}(\mathcal{H}_1) = \textsc{Canon}(\mathcal{H}_2)$ *iff* $\mathcal{H}_1 \cong \mathcal{H}_2$.

IR proceeds by iterated color refinement to a stable coloring; when refinement leaves a non-singleton cell, it *individualizes* one vertex (assigns it a distinguishing color) and recurses, exploring a search tree of refinements. Automorphisms discovered along the way prune the tree so that symmetric branches are not re-explored, and the lexicographically extremal leaf defines the canonical labeling.

Remark 3.2 (Complexity)**.** *Like nauty, IR is exponential in the worst case and terminates in time polynomial in the state size on instances without large automorphism groups. It gives no polynomial-time guarantee, and we claim none. The worst case is realized on highly symmetric states, which is where the exact mode's cost is concentrated. No cheaper exact alternative is available to substitute for it: Proposition 3.3 bounds what a Weisfeiler–Leman pre-filter can remove, and Section 8 reports the two implemented alternatives, one measured slower and one that never fires.*

The automorphism generators and edge orbits produced by IR are reused elsewhere in the engine: they define the canonical edge-orbit equivalence used for principled event canonicalization and for exact reconstruction of causal and branchial multiplicities.

Two bounds, and only one of them is evident when it is exceeded. The search is bounded twice: by the individualization depth it may descend to, and by the number of automorphism generators it may record. Exceeding either is reported to the caller, which raises the bound and re-runs, so neither is a silent truncation. The two are not symmetric in what a truncation would cost. A depth bound reached before a discrete coloring means no canonical form was produced at all, and the failure is evident. A generator table filled to capacity, by contrast, yields a canonical form that is correct: the hash is exact regardless of how many generators were recorded. What degrades is the orbit partition, which is obtained by fusing edges over the generators found—fewer generators fuse fewer edges, so the orbits come out too *fine*. Since the orbits are how the quotient reconstruction identifies instances (Section 5.10), a short generator table produces a wrong identification rather than a slow run, and it produces one that looks like an ordinary answer. This is why the generator bound escalates on the same footing as the depth bound rather than being treated as a performance parameter. More generators cannot repair a depth failure, so escalation stops at the first depth report.

The canonical labeling of a symmetric state is a coset. When the automorphism group is nontrivial, IR does not determine a single labeling but a coset of them, and the algorithm's within-cell tie-break selects one representative. That tie-break reads the vertex numbering it was handed. Two conventions for numbering a state's vertices therefore select two different representatives, each internally consistent and both agreeing on the canonical *hash*, while disagreeing about which edge receives which rank. Ranks are what identify consumed and produced edges across a rewrite, so the convention has to be fixed once and shared by every caller rather than chosen locally: measured on the equivalence probe's 4063-state corpus, encounter-order numbering and sorted-unique numbering disagree on 103 states, all of them states with nontrivial automorphisms.

3.2 The Weisfeiler–Leman Signature

The engine also computes a 1-dimensional Weisfeiler–Leman (1-WL) color-refinement hash [3, 9]. WL is a sound but incomplete invariant: isomorphic states always receive the same WL hash, while certain non-isomorphic states — regular graphs with identical local structure, for instance — collide. It is the identity used by the modes that request a coarser notion of state equality. It is not used by the exact mode, for the reason established in Proposition 3.3.

Proposition 3.3 (Ceiling on a WL pre-filter)**.** *Let a run produce R raw states falling into C isomorphism classes. Consider deduplication that computes the WL hash of each raw state, and invokes IR only when that hash has been seen before. Then the number of IR invocations is at least $R - C$, so the fraction of IR calls avoided is at most C/R, and one WL pass is paid on all R states.*

Proof. WL is isomorphism-invariant, so states in the same class share a WL hash and the number of distinct WL hashes is at most C. A state whose WL hash is already present cannot be resolved by WL: equal hashes are consistent both with isomorphism and with a WL collision, so IR is invoked. At most one state per distinct WL hash finds its hash absent, hence at most C states skip IR and at least $R - C$ invoke it. □

The bound is only useful with C/R in hand, and that ratio is a property of the rule and the depth. Measured across the rule-type corpus, reported per case beside the event count, since the two must be read together (Table 1, plotted in Figure 2):

Table 1: **Canonical-class to raw-state ratio, per corpus workload.** C/R is the fraction of raw states that are distinct up to isomorphism, reported beside the event count because the two must be read together: a small ratio on a small run says less than the same ratio on a large one. This is the quantity Proposition 3.3 bounds the achievable saving by. Source: `tools/cost_matrix.cpp`.

Case	Rule type	Events	C/R	C/R at lower depth
`cycle4-automorphic`	automorphism	68,184	0%	12%
`star4-automorphic`	automorphism	68,184	0.3%	20%
`disconnected-lhs`	disconnected	126	0.8%	14.3%
`binary-growth`	productive/arity2	873	4.1%	75%
`multi-rule`	multi-rule	237	15.1%	83.3%
`triangle`	productive/mixed-conn	124	28.8%	60%
`arity3-growth`	arity3	153	42.2%	100%
`self-loop`	self-loop/arity2	1	100%	100%

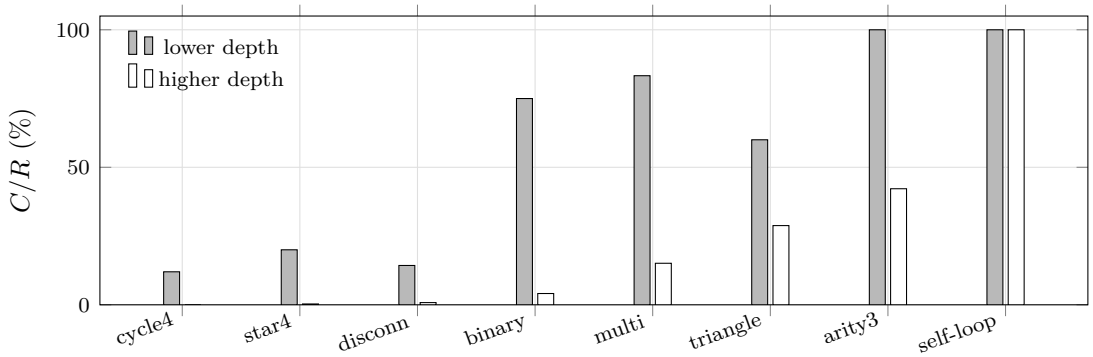

Figure 2: C/R, the ceiling of Proposition 3.3 on the fraction of IR invocations a Weisfeiler–Leman pre-filter could elide, for each corpus case at two depths. The bar pairs are two depth samples per rule and not a continuum, so the figure shows direction rather than a rate. Every case that continues to rewrite decreases. A case that does not decrease is one whose two samples produced the same class-to-raw ratio, which happens where the rule reaches a fixed point before the higher depth—`self-loop` does so after one event, and its ratio is 100% at both. Values as in Table 1.

For every rule that continues to rewrite, C/R decreases with depth. Reaching the same state along several distinct histories is characteristic of multiway evolution, and each such arrival produces a repeated WL hash that IR must resolve. The cases that retain a high ratio are those reaching a fixed point after one or two steps: `self-loop` has $C/R = 100\%$ over two raw states and one event. The fraction a filter could elide is therefore largest on the workloads with the least work to elide.

Two options require automorphism data: quotient exploration, and the event canonicalization conventions of Section 5.14. Both obtain the edge orbits from the same IR pass, so no second canonicalization is performed.

Algorithm 1 Weisfeiler–Leman Hash

procedure WLHash($\mathcal{H}$)
  Initialize $c_v^{(0)} \leftarrow \deg(v)$ for all v
  $i \leftarrow 0$
  repeat
    $i \leftarrow i + 1$
    for each $v \in V$ **do**
      $c_v^{(i)} \leftarrow \text{Hash}(c_v^{(i-1)}, \text{multiset}\{c_u^{(i-1)} : u \in N(v)\})$
  until $\{c_v^{(i)}\} = \{c_v^{(i-1)}\}$ ▷ partition stable
  return Hash(sorted($\{c_v^{(i)} : v \in V\}$))
end procedure

Proposition 3.4 (WL Complexity)**.** *Algorithm 1 gives the refinement. 1-WL requires $\mathcal{O}(k \cdot |E|)$ time per refinement round for colours of size k, with at most $|V|$ rounds, so $\mathcal{O}(|V| \cdot k \cdot |E|)$ overall (Appendix A).*

This polynomial bound is why WL remains available for the coarser modes. It is not an alternative canonicalizer: where exactness is required, IR (Section 3.1) is the only identity used.

3.3 State Canonicalization Modes

Rather than exposing a choice between competing hashing algorithms, the engine exposes a single *state canonicalization mode* that controls how aggressively isomorphic states are merged:

```
enum class StateCanonicalizationMode {
    None,        // keep all states distinct; no dedup
    Automatic,   // content-ordered hash; identifies literal edge
        content
    Full         // exact: the IR canonical hash, on every state
};
```

Listing 1: State canonicalization mode

None performs no deduplication and keeps every raw state distinct. Automatic deduplicates by a content-ordered hash of the edge tuples as given. This hash is not an isomorphism invariant: it identifies two states when their edge tuples are literally equal, and separates isomorphic states whose vertices carry different identifiers. Full is the exact mode used for all correctness-critical results: the IR canonical hash is computed for every state and is itself the deduplication key, so two states are identified if and only if they are isomorphic.

Remark 3.5 (Incremental hashing)**.** *Child states differ from their parents by only the consumed and produced edges, and the engine passes that per-event delta to the canonicalizer. The hash is nonetheless computed from the state's complete edge set in every mode. No incremental-hashing result is reported here, and none of the measurements in Section 8 depends on one.*

4 Pattern Matching

Pattern matching—finding all instances of a rule's left-hand side in a hypergraph—accounts for a large fraction of evolution time; the instruction profile of Table 17 attributes 24.8% of executed instructions to matching and its join—the largest named component, though the unattributed remainder is larger still. We use multi-level indexing to restrict candidate enumeration.

4.1 Edge Signatures

Definition 4.1 (Edge Signature)**.** *The* signature *of hyperedge* $e = (v_1, \ldots, v_k)$ *is the pattern* $\sigma(e) = (s_1, \ldots, s_k)$ *where* $s_i = s_j$ *iff* $v_i = v_j$.

For example, edge $(3, 3, 7)$ has signature $(0, 0, 1)$, capturing the repetition pattern without specific vertex labels. Pattern edges can match data edges only with compatible signatures. For pattern signature σ_p with d distinct variables, the number of compatible data signatures is the Bell number B_d [20]; Table 2 gives the first few. The host tests each candidate edge's signature against the pattern's directly; the device precomputes the compatible set per pattern edge (Section 6).

Table 2: Bell numbers for signature compatibility

d	1	2	3	4	5	6	7	8
B_d	1	2	5	15	52	203	877	4140

4.2 Candidate Edges by Ancestry

A state is a view into the shared edge pool (Section 5.2), and its matching asks two questions of that view: which of its edges contain a given vertex, once a pattern variable is bound, and which of its edges carry a compatible signature, for the seed. Both are answered from the state's ancestry. Every edge of a state was brought by exactly one state on its chain of parent states: the root brought all of its edges, and each derived state brought the edges its event produced. Each state indexes its own contribution by vertex once, when it is created, so the edges of S that contain v are

$$\bigcup_{A \in \text{chain}(S)} \{e \in \text{Contrib}(A)[v] : e \in S\}, \tag{1}$$

where $\text{Contrib}(A)[v]$ is A's sorted incidence list for v and membership $e \in S$ excludes an edge that a later event on the chain consumed. A query costs the chain's length times one search of a small sorted array, nearly every edge it tests is in the state, the index of a derived state is the size of its event's right-hand side, and nothing is maintained per edge when an edge is created. A global index over the whole evolution's edges, filtered per state, answers the same question at a cost proportional to every edge ever created at v in any state, of which a state at depth seven holds under two percent.

4.3 Matching Algorithm

Pattern matching proceeds through the task-based system of Algorithm 2:

The order $e_1, \ldots, e_k$ is fixed per rule when the rule is added, and it is required to be connected—every edge after the first shares a variable with one already bound—so no step of the join is a cartesian product. Two host paths execute that order: a synchronous depth-first join, which runs an entire search inside one task and spawns a job only for a completed match, and the task-spawning form above, which spawns a task per candidate and is what exposes the

Algorithm 2 Task-Based Pattern Matching

```
1: procedure FindMatches(L, H)
2:     e_1, ..., e_k ← MatchOrder(L)                ▷ connected: no step is a product
3:     Enqueue ScanTask(e_1, ∅)
4:     while task queue non-empty do
5:         τ ← Dequeue()
6:         if τ is ScanTask(e_i, φ) then
7:             for each e ∈ S with σ(e) compatible with σ(e_i) do
8:                 if Compatible(e, e_i, φ) then
9:                     Enqueue ExpandTask(e_{i+1}, φ')
10:        else if τ is ExpandTask(e_i, φ) with i > k then
11:            yield complete match φ
12:        end if
13:    end while
14: end procedure
```

search itself to work stealing. They are not two joins. The recursion, the edge-injectivity check, the binding and unwind, and the rule choosing which pattern position is bound next are one shared body, which is also the body the device runs; the two paths differ in who executes the resulting steps and when.

4.4 Position Against the Worst-Case-Optimal Bound

Treating a left-hand side as a conjunctive query, the AGM bound [21] gives the maximum number of results a query can have on a database of a given size, and a join is worst-case optimal if its running time matches that bound up to logarithmic factors [22]. Our join is edge-at-a-time: it binds one pattern hyperedge per expansion, in the connectivity order of Section 4.3.

For left-hand sides of one or two hyperedges, an edge-at-a-time join meets the bound, and every rule in the ORACLE corpus of Section 8 is of that form; the left-hand-side sweep of Section 8.8 deliberately goes past it, to three and four edges. It does not meet the bound in general. On a cyclic left-hand side of three hyperedges the AGM bound is $N^{1.5}$ for a state of N edges, while an edge-at-a-time join performs $\Omega(N^2)$ work: two of the three edges can be bound before the third constrains anything, and the intermediate result is already quadratic. No choice of join order removes this, since the difficulty is the granularity of binding rather than its sequence. Closing it requires enumerating variable-at-a-time in the manner of a worst-case optimal join, which in turn requires indices supporting ordered seek on a bound variable rather than the incidence lists described above.

We therefore state the position rather than claim the property: the join is worst-case optimal on the rule shapes measured here, and is not worst-case optimal in general. The static analysis of Section 5.6 computes, per rule, whether its left-hand side is acyclic and what its integral edge cover is, so a rule set for which this gap matters is identifiable before it is run.

4.5 Match Forwarding

When state s' is derived from s, a match valid in s remains valid in s' unless it used an edge the rewrite removed. The engine forwards such matches from parent to child rather than rediscovering them:

Proposition 4.2 (Match Forwarding)**.** *Let $\mathcal{M}$ be a match in state s, and let s' be derived from s by applying rule r at match $\mathcal{M}'$. Then $\mathcal{M}$ remains valid in s' iff $edges(\mathcal{M}) \cap edges(\mathcal{M}') = \emptyset$.*

Forwarding therefore avoids re-running the join for every match a child inherits unchanged from its parent.

Delivering it under concurrency requires two mechanisms rather than one. A match discovered in a state is *pushed* to the children that state already has, which covers every child registered at the time of discovery. A state created later cannot receive a push that has already happened, so on creation it *pulls*, walking the chain of ancestors and collecting each one's matches that its own rewrite did not invalidate. Neither half suffices alone: without the push, every state would repeat its ancestors' work on creation; without the pull, a match discovered before a child existed would never reach it.

The two halves interact through an ordering requirement. The pull's ancestor walk treats a missing parent link as having reached a root and stops there. A child must therefore publish its parent link *before* it becomes visible in its parent's child list: once it is visible it may receive a forwarded match, that match may create a grandchild on another worker, and that grandchild's pull walks upward through this child. Were the link published after visibility, such a walk could find it absent, stop early, and miss every match held only in higher ancestors—without any indication, because pulled matches are not re-stored in the descendant and pushes happen once at discovery, so nothing later repairs the omission. Publishing the link first makes an absent link mean *root* for every walk that can reach the state.

Forwarded matches are accumulated per parent and delivered to the child together rather than one at a time, and whether forwarding is applied at all is decided per rule set by the static analysis of Section 5.6; Section 8 reports the ablation.

4.6 Delta Matching

Forwarding supplies a child with the matches it inherits unchanged from its parent. Its complement is forced: a match present in the child but not in the parent must use at least one produced edge, because a match built entirely from edges that survived the rewrite was already a match in the parent. The child's own scan is therefore restricted to matches anchored on a produced edge, which is a small set—one rewrite produces the right-hand side's edges, independently of how large the state is.

The two mechanisms partition the child's matches rather than overlapping: every match either touches a produced edge or does not, forwarding covers the second class and the delta scan the first. What does overlap is the delta scan with itself. A match touching k produced edges is discovered k times, once with each of them as the anchor, so the results are deduplicated on content rather than on the anchor that found them; duplicates here are routine rather than exceptional, so the deduplicating lookup precedes the allocation of a match record rather than following it.

Because the partition is an argument rather than a measurement, the engine carries a validation mode that checks it. With the mode enabled, every state's matches are additionally computed by an unrestricted scan and compared against the union of what forwarding and the delta scan supplied, and a missing match is attributed to the obligation that owed it: to forwarding if it uses only surviving edges, to the delta scan if it touches a produced edge. The two have different causes and different repairs, so the attribution is recorded separately rather than as a single mismatch count. The validator also counts its own executions, since a mismatch count of zero is evidence only when the number of comparisons performed is not.

5 System Architecture

The architecture follows from two commitments that constrain every structure below. The first is that a rule—an ordering, an identity, a hash, a predicate—has exactly one implementation, compiled for both the host and the device, so the two targets cannot diverge on what a state *is*. The second is that no thread ever blocks another: every shared structure is lock-free, and where a protocol needs agreement it reaches it by a single compare-exchange whose loser adopts

the winner's answer rather than waiting. The subsections take these in turn—the shared core, the storage the states live in, the lock-free structures, the allocator that keeps them off the global heap, the memory model that makes their orderings checkable, and the static analysis that decides how a rule will be matched before any matching happens.

5.1 One Implementation per Rule, Compiled for Both Targets

The CPU engine and the CUDA port are separate schedulers over the same semantics, and any rule implemented twice will eventually be implemented differently. Every decision procedure the two share therefore exists once, as an annotated function compiled for host and device from a single source: the join that enumerates matches, the edge-signature rule, the rewrite semantics determining which vertices a produced edge carries, the event identity, the Weisfeiler–Leman hash, the individualization–refinement canonical hash, and the quotient-causal dynamic program and its per-instance replay. Eleven such cores are shared in total; the three not named here are the matcher's task core, the sampling draw and the slot rule.

The consequence is that host and device agree by compilation rather than by testing, and a differential test between them can be reserved for the parts that genuinely differ—scheduling, memory placement and queueing—rather than re-checking a rule that exists once. What the arrangement rules out is a rule written twice: two implementations of one decision agree until one of them is edited, so the duplication is the defect and not the disagreement it eventually produces.

5.2 Unified Hypergraph Storage

Our core data structure stores all hypergraph data in a unified edge pool, shown in Figure 3: every state is a bitset view over one shared pool of edges, so a child state that differs from its parent by a few edges stores that difference and not a copy.

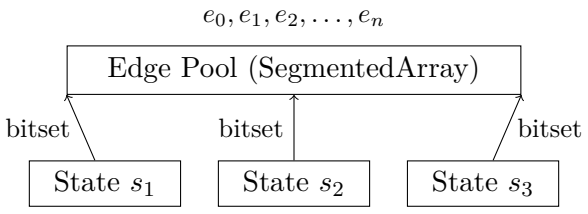

Figure 3: Unified edge storage with state views

This design eliminates edge duplication across states, since each state is a bitset view over the shared pool rather than an independent copy of its edges. The bitset is chunked and the chunks are shared: a child state initially references its parent's chunks, and a chunk is copied only when the child first mutates it. A rewrite that touches a few edges therefore allocates a few chunks rather than a copy of the parent's membership, and chunks are immutable once shared, so concurrent readers of a parent are unaffected by a child's mutation. Table 5 reports the arena bytes each corpus workload consumes.

5.3 Lock-Free Data Structures

Concurrent evolution requires thread-safe data structures without lock contention. The constructions below follow standard non-blocking techniques [23], in particular compare-and-swap insertion into an open-addressed table [24] and lock-free list linkage [25]:

ConcurrentMap⟨K, V⟩: Hash table using atomic compare-and-swap for insertion. Wait-free lookup with lock-free insertion.

ConcurrentKeySet⟨K⟩: A set, for membership rather than mapping. Presence and ownership are decided by one compare-exchange from the empty sentinel to the key, so none of the map's value-publication protocol exists: no locked slot, no settling of a claim met mid-flight. An insert scans the superseded-table chain for a settled copy of its key before claiming at the head, and a claim that lands in a table which is no longer the head is reported stale and re-driven rather than counted—the two halves of keeping one key from ever being claimed twice. Growth carries each key into the new table and then seals the slot it came from with a second reserved key, so a key reachable through two tables is enumerated once, and a table whose slots are all sealed is skipped rather than probed—without which a retired table is scanned in full on every operation, because sealing removes the empty slot linear probing stops at. Membership queries walk the chain oldest table first, and Section 7's reader-visibility harness checks that order. The quotient reconstruction's membership marks use it (Section 5.10).

SegmentedArray⟨T⟩: Dynamically growing array stored in fixed-size segments. Append-only with atomic size tracking.

LockFreeList⟨T⟩: Append-only linked list for index entries. Supports concurrent iteration via snapshot semantics.

Work-stealing deque: A bounded Chase–Lev deque [26]. Each worker owns one, pushing and popping at its own end without a contended atomic in the common case, and stealing from the opposite end of another worker's deque when idle. Its memory orderings follow Lê et al. [27]. Following HGMatch [12], evolution is expressed as a dataflow over such a scheduler: a scan task seeds a join, expansion tasks extend it, and completed matches are handed to rewriting, with work stealing providing load balance across a frontier whose width is not known in advance.

Job pool: Tasks are drawn from a per-thread slab rather than allocated, so submitting a task performs no call to the global allocator, so the allocation figures of Section 5.4 hold for a workload that submits one task per candidate expansion. Allocation and same-thread release touch only the owning thread's private free list, with no atomic and no contention. Work stealing means a task allocated on one thread is often released on another, so a foreign release is pushed onto the owner's separate foreign list by compare-and-swap, and the owner reclaims the whole accumulated batch with one exchange when its private list empties. That shape is free of the ABA problem by construction rather than by a tag: producers only ever prepend, and the single consumer takes the entire list at once, so there is no node for a compare-and-swap to observe unchanged after reuse. Each slot carries a header naming its owning pool, so a release on any thread reaches the pool the slot came from; an object too large for a slot falls back to the general allocator, so correctness does not depend on the slot size being estimated correctly. Pools are never destroyed and are recycled through a fixed array of claimable slots when a thread exits—an array rather than a stack of free pools, because a pool that is claimed, used and released again is exactly the reuse a stack's head compare-and-swap cannot distinguish, while an array slot has no pointer to compare.

The shared injector, and what bounds its ABA defence. Work submitted from outside a worker, and work that overflows a worker's own deque, goes to one shared double-ended queue. Its state is a single 64-bit word packed as a 32-bit tag and two 16-bit indices, so one compare-exchange commits both ends at once and a pop at each end racing for the last item

resolves by one of the two exchanges failing. The three fields share a word because a 128-bit compare-exchange is not lock-free on the supported targets; the indices need 16 bits each for a capacity of 32,768 entries, which leaves 32 bits for the tag. Because both ends move in both directions, the index pair recurs, and the tag lets a compare-exchange commit only if nothing changed since the load.

A 32-bit tag wraps, so the defence is conditional, and the condition is stated here rather than assumed. For a stale item to be claimed twice, the tag must complete a full cycle *and* both indices must return to the values one popper read, all within that popper's own load-to-exchange window—a handful of instructions, so it requires the thread to be descheduled inside them. The throughput measurement that bounds this is direct: at eight threads the shared queue sustains 8.16×10^6 operations per second, so 2^{32} of them take on the order of 500 seconds of continuous stall while every other thread runs flat out. The same measurement gives the cost of sharing one word: 1.33×10^8 operations per second at one thread against 8.16×10^6 at eight. Per-worker deques are near-exclusive to their owners and do not incur that reduction; the injector is shared and does, so it holds only overflow and external submissions.

Neither queue offers a blocking push or pop. For a worker this is not a preference but a requirement: a worker parked inside a push cannot pop, so if every worker parked there the queues would never drain again—and since expansion is combinatorial and submits from inside running jobs, saturating a local deque and the injector together is reachable rather than hypothetical. A full queue is answered by running the job on the calling thread, which always makes progress.

A worker with no available work does not spin indefinitely. It waits on a 32-bit address through the platform's own primitive—`futex` on Linux, `WaitOnAddress` on Windows, `os_sync_wait_on_address` on macOS—so a blocked worker consumes no cycles and is woken by the thread that supplies the work. No mutex or condition variable is used on any of these platforms: the engine's object files reference none, and the only such construction in the tree is a fallback compiled solely for a platform providing none of the three primitives.

One code path for a machine with no threads. The scheduler also runs with no worker threads at all. In that mode none are spawned: submission routes every job to the injector and the wait call drains it inline, in first-in-first-out order, on the thread that called it. Jobs submitted by a running job land behind it and execute in submission order, so a run in this mode is deterministic by construction rather than by the argument of Section 5.8. This is the single-threaded execution mode, and it is also the WebAssembly target, where spawning threads is not available. Nothing above the scheduler distinguishes the two: the engine is compiled once and the work the workers would have done happens on the thread that waits.

5.4 Allocation

Evolution allocates from per-worker arenas rather than from the global allocator. Each worker holds a bump cursor into blocks it owns, so allocation on the matching and rewriting paths is a pointer increment with no synchronization, and memory is reclaimed in bulk by resetting the arena rather than by freeing individual objects. Calls to the global allocator are consequently confined to acquiring those blocks: Table 5 reports 55 to 218 such calls for an entire run, which on the largest workload measured is 0.0032 calls per event.

5.5 Memory Model

A new state is published to readers by a release store paired with an acquire load, so a reader that observes the state also observes the edge set written before it:

```
// Producer: creating new state
edges_.append(new_edge);           // seq_cst store
```

```
state.edge_set.insert(edge_id);
std::atomic_thread_fence(std::memory_order_release);
states_.append(state);            // release store

// Consumer: accessing state
StateId id = canonical_map_.lookup(hash);  // acquire load
std::atomic_thread_fence(std::memory_order_acquire);
State& s = states_[id];           // safe to read edge set
```

Listing 2: Memory synchronization example

5.6 Static Rule Analysis

Some properties of a rule set are decidable from the patterns alone, before any evolution runs, at the cost of a bounded scan over two finite patterns. The engine computes them when a rule is added and uses them to select strategy.

Per rule it records the edge-count difference between the two sides, which predicts whether states grow and therefore how canonicalization cost will develop; the number of variables the right-hand side introduces, since a rule creating none operates within a bounded vertex set; and the left-hand side's arities. Treating the left-hand side as a conjunctive query, it computes whether that query is acyclic—which determines whether a join order exists that avoids intermediate blow-up—and an integral edge cover, which bounds the number of matches a state of N edges can yield by N^{ρ}. That bound is tight exactly when the query is acyclic; where it is not, the recorded value is an upper bound and is used only as one.

Every predicate is sound in one direction only, and each is stated with its direction. The branching test over-approximates: a negative result means branching is structurally impossible, while a positive result means only that it has not been ruled out. An adaptation may act on the negative and must not act on the positive. The engine does exactly that with the branchial relation: if no two matches can share a consumed edge then no state has a branchial pair under any initial condition, so the relation is provably empty and is not built. The request is not overridden by this—what the caller asked for is the empty relation and the empty relation is what it receives; only the work of discovering it is removed. Nothing here concerns termination, finiteness of the reachable set, or global confluence, all of which are undecidable for a rewriting system of this power.

5.7 Behavior at Configured Limits

The engine's containers are sized from configuration, and a workload may exceed them. Reaching a configured ceiling is treated as a distinct outcome from a programmer error: the run returns the portion of the evolution that fits, together with a warning naming the limit that was reached, rather than raising an exception and discarding the work. Both backends behave identically in this respect. Options that a build does not implement are reported the same way, so a caller who requests one receives the result computed without it and a statement to that effect, rather than silently receiving a different computation from the one requested.

5.8 Determinism Contract

The engine's observable output—the state set, the event set, and the causal and branchial relations—is a deterministic function of the rules, the initial state, the step count and the options. It does not depend on the number of worker threads, on the order in which the scheduler happens to execute tasks, or on whether quotient exploration is enabled. Identifiers assigned to states and events are allocated in arrival order and therefore do vary between runs; every observable defined above is stated over canonical identities and is invariant. This property is

checked by a determinism gate that fingerprints the expanded state set, the (input, output, rule) transition multiset, and the causal and branchial relations across thread counts and seeds, and on a divergence reports which component moved and under which configuration. Reporting the components separately makes the gate sensitive: the causal relation can move on its own, by a single edge, while the state set, the transition multiset and the branchial relation stay identical, so a gate that compared one aggregate fingerprint would not distinguish that from agreement.

The capping options are outside it, for the reason that shapes the sampling design: MaxStatesPerStep admits states until a per-step counter reaches its bound, so which states are admitted follows the order they arrive in and varies with the worker count. An exact cap that was also schedule-independent would have to hold every candidate at that depth until the depth settled, which is the barrier Section 5.12 declines: only a population that completes locally can be finalised without one, and a per-step population does not. What a capped run guarantees is therefore the bound, at every worker count, and not which states meet it. The per-parent cap has a locally-completing population and could be made schedule-independent on the same terms as the transition spine; it is not today.

5.9 Depth as a Shortest Path, and Termination

A run is bounded by a number of steps, and the bound is applied to the *shortest* path length from an initial state to a canonical state rather than to the length of the path along which the state first happened to be reached. The distinction has no content sequentially, where the search reaches states in order of increasing depth. It has content under concurrency: a state can be reached first along a long path and only later along a short one, so a budget applied to arrival depth would admit or exclude states according to which path won a race, and the set of states expanded would vary with the thread count.

The engine therefore *relaxes*. When a state is reached along a path shorter than its recorded depth, the recorded depth is lowered and the reduction is propagated into the state's children, which may in turn be lowered, and so on. The cascade terminates because depth strictly decreases on each accepted relaxation and is bounded below by zero. Matching is not repeated by it: the claim that admits a canonical state to expansion is a separate value from its depth, so a state whose depth falls is matched at most once regardless of how many times it is relaxed.

Relaxation and child registration race, and the race is settled by ordering rather than by exclusion. Registering a child publishes it into its parent's child list and then reads the parent's depth; relaxing a parent lowers the parent's depth and then scans the parent's child list. Sequentially consistent fences on both sides exclude the one outcome that would strand a child at a stale depth—the scan missing the child *and* the read missing the lowered depth—so a child always learns its parent's true current minimum, whichever side of the race it is on. A state that lies past the budget is retained as a frontier state for a continuation to resume from rather than discarded, and its expansion claim is deliberately left untaken, because a shorter path found later in the same run must still be able to relax it into budget, and a state that has already been claimed never would.

Budget consumption is claimed by compare-and-swap rather than by an unconditional increment followed by a rollback on overshoot. The two differ in what other threads observe: a rollback publishes a count above the limit for as long as it takes to undo, and the readers that prune work against the budget read exactly that count, so N concurrent claimants would make each other's in-budget work appear out of budget and which work was pruned would depend on the interleaving. A compare-and-swap claim never publishes a count above the limit, so the budget prunes the same work under every schedule.

Termination is detected without a barrier. A depth is settled when it holds no live work and the depth below it has already settled; settling one depth cascades to the depth above, which may have been waiting only on it. The argument that these two conditions suffice is that a task at depth d submits work only at depths greater than d, so once $d-1$ is settled nothing can place

work at d again. No thread waits on any other, and the check is a test of two values rather than a rendezvous.

5.10 Quotient Exploration

Under full expansion, every raw state that is produced is itself expanded, so a state reachable along k distinct histories is matched against k times. Because the canonical hash already identifies these as one state, that work is redundant for the purpose of discovering new states.

Quotient exploration retains the events but not the repeated expansion: a state found equivalent to one already seen is still created, and its incoming event is still recorded, so the event set and the causal and branchial relations are unchanged; what is suppressed is the spawning of match tasks from that state. Exploration therefore proceeds from one representative of each isomorphism class, at the shallowest depth at which the class was reached. Table 7 reports raw states expanded and wall time for the two modes at increasing depth.

The two modes agree on the canonical state set by construction. That they also agree on the causal and branchial relations is less immediate, because those relations are defined over raw events and quotient exploration expands fewer states.

The reconstruction is a forward tally. Under an exact identity the canonical states graph is cyclic, since a canonical state recurs at several path lengths; its unrolling by depth is acyclic, and the multiplicity of each (canonical state, depth) pair satisfies

$$\mathrm{mult}[s_0, 0] = 1, \qquad \mathrm{mult}[c, k{+}1] \mathrel{+}= \mathrm{mult}[s, k] \cdot w(s \to c),$$

summed over the transitions into c, with the raw event count recovered as $\sum \mathrm{mult}$. Every quantity propagates forward over the unrolled DAG, so a *count* is recovered by summation rather than by re-exploring, and that part is polynomial in the number of (canonical state, depth) pairs.

Recovering the raw *observables* is a different matter, and the distinction is the one that decides what quotient exploration costs. Mapping a multiplicity back to individual instances requires distinguishing the edge roles within a class, which the edge orbits from the same IR pass provide (Section 3.1); no additional canonicalization is performed. But the instances are then materialised one by one, and there are as many of them as the full system holds. On a two-rule set whose canonical state count grows linearly with depth—three, four, five, six, seven, eight states at depths two through seven—the raw system reached 146,599 states by depth six, and the reconstruction's cost tracked that rather than the canonical count. Quotient exploration visited 49 raw states there against full capture's 146,599 and still ran only 65 times faster: a large constant factor, not the asymptotic saving the state counts suggest.

Quotient exploration and raw reconstruction are separate requests, and the implementation keeps them separate. The reconstruction runs only when the causal relation, the branchial relation or the raw event set is among the requested outputs, so a run that asks for canonical states alone performs neither the materialisation nor the growth that accompanies it. The reconstruction is also not a single-threaded pass—every worker drives it, and its total cycle count grows with the thread count, which Section 8.16 records among the measured limits.

5.11 Transitive Reduction of the Causal Graph

The causal graph accumulates an edge for each dependency between events, including those implied by longer paths. The transitive reduction removes an edge $p \to c$ whenever a directed path $p \rightsquigarrow c$ of length greater than one already exists; on an acyclic graph this reduction is unique.

Maintaining a transitive reduction under edge insertion is an instance of the dynamic transitive-reduction problem. Goranci et al. [17] give the first fully dynamic algorithms for maintaining a transitive reduction of a general directed graph under both insertions and deletions,

achieving $\mathcal{O}(m + n \log n)$ amortized update time, and $\mathcal{O}(m + n^{1.585})$ worst case using fast matrix multiplication. The setting here is strictly easier in two respects: causal edges are only ever inserted, never deleted, and they arrive in an order consistent with a topological order of the causal DAG, since an event's producers are complete before the event exists. We therefore make no claim of novelty for the procedure below, and no claim of optimality; it is the specialisation the easier setting permits, and a fully dynamic reduction would be required if deletions were admitted.

Why the reduction is maintained rather than computed. Computing the reduction as a pass over the finished graph would require the graph to be finished, which conflicts with two properties the engine offers. A session (Section 5.13) may be queried between steps, so there is no point at which the causal graph is known to be complete; and the reduction is what most consumers of the causal graph actually want, so materialising the full relation first and reducing afterwards would allocate the transitive closure that the reduction exists to avoid. The reduction is therefore maintained as an invariant: after every insertion, the retained edge set is the transitive reduction of the causal DAG seen so far.

Procedure. Let $e = (p, c)$ be a new causal edge, meaning event p produced an edge that event c consumed. Insertion asks whether c is already reachable from p through the retained edges; if it is, e is implied and is not retained, and if it is not, e is retained. There is no removal step; the arrival discipline makes one unnecessary. A consumer's incoming edges are registered by that consumer's own worker in descending producer-event order, and an ancestor's id is strictly smaller than its descendant's. A retained (p', c) could become implied only through a path $p' \to \cdots \to x \to c$; if (x, c) was registered before (p', c) then the path was already visible to the arrival query, which drops (p', c) on the spot, and if after, then $x < p'$ in the descending order while reachability from p' requires $p' < x$—a contradiction. An edge accepted at arrival therefore stays non-implied forever, and deciding each edge once maintains the exact reduction. The descending order also makes the queries cheap: the closure reachable from nearer producers is established before farther producers are tested against it, and a farther producer is usually eliminated by a query that terminates early. Because events are numbered in creation order and an event's producers necessarily precede it, the order is available without sorting.

What the arrival order establishes. Two properties of this setting make the problem strictly easier than the fully dynamic one. Causal edges are only inserted, never deleted, so no retained edge can become non-implied later, and an edge dropped as implied never has to be reconsidered. And edges arrive in an order consistent with a topological order of the causal DAG: an event exists only once its producers do, so when (p, c) arrives, every edge into p has already been processed and the closure below p is final. A fully dynamic algorithm must maintain reachability under deletions, and that requirement forces the $\mathcal{O}(m + n \log n)$ amortized bound of Goranci et al.; neither of the two properties above is available to it.

Order independence. The retained edge set is required to be a function of the causal DAG alone and not of the order in which edges were inserted. This is not automatic: a concurrent evolution inserts causal edges from several workers, and a reduction that depended on interleaving would produce different graphs on different runs of the same input. The requirement is the same schedule-independence property as Section 5.8 and is checked by the same gate, which compares the reduced edge set across thread counts and seeds. It is secured differently on the two paths that serve the relation. Full capture holds the arrival discipline of the procedure above—each incoming edge registered by its consumer's own worker, in descending producer order, against ancestors whose own registration is complete—and under it the drop-at-arrival decision is exact whatever the interleaving of consumers. The quotient reconstruction emits

between canonical event ids, which are not monotonic under recurrence, so no arrival discipline exists there; it stores the relation unreduced and reduces on read, where the uniqueness of a finite DAG's transitive reduction makes the answer a function of the stored set alone.

The reduction applies only to the causal graph. States, events and the branchial relation are never reduced: branchial edges connect states at the same step and carry no transitivity to remove.

5.12 Sampled Exploration

Exhaustive evolution is not always the object of interest, and the state count grows too quickly for it to be available at large depth. The engine therefore admits sampling, applied at the transition rather than at the state: each candidate transition is retained with a fixed probability, so the number of children a state contributes remains a random variable rather than being truncated to a fixed count. Truncating to a fixed count per state would collapse the offspring distribution to a point mass and remove the property the sample exists to estimate.

Per-rule weights multiply that retention probability, so a rule may be explored fully while another is suppressed entirely, and the two controls compose rather than one overriding the other. A seed makes a sampled run reproducible; sampling decisions are keyed on canonical identity, so the same seed yields the same sample at any thread count, consistent with Section 5.8.

Choosing the rate. A fixed retention probability applied at every depth is not a neutral choice, because the expected number of surviving children per state compounds with depth. On the workloads measured, retention of 1/8 and 1/4 extinguish the frontier within the first step, while 1/2 leaves the population growing; the useful range between extinction and no thinning is narrow and depends on the rule's branching. The rate is therefore permitted to vary with depth, so that a sample can be thinned without the frontier dying before it reaches the depth of interest.

Coverage is limited by reachability, not by survival. A per-transition coin does not recover the exhaustive system as seeds are accumulated, and the reason is structural rather than a matter of the rate. A transition can only be sampled if its source state was created, and its source state was created only if some earlier coin fired, so what a run misses is not a scattering of individual transitions but every subtree below a transition that was dropped. Two attempts to improve coverage by adding independent randomness—an extra coin per arrival depth, and an extra coin per arriving ancestor class—left the union-over-seeds recovery curve unchanged when measured, which is the signature of a limit on reachability rather than on per-transition survival.

The control that does move that curve keeps a connected skeleton. A cap of k transitions per (state, rule) pair retains at most k of a state's own transitions for each rule, selecting them by a deterministic pseudorandom rank of the canonical transition key mixed with the run's seed. Every state that is reached therefore keeps at least one outgoing transition, so the sample is a spanning substructure rather than a thinned scattering, and a different seed keeps a different skeleton—so the union over seeds grows in reachability, which is the quantity that was limiting.

The selection is made when the state's matching completes rather than as matches arrive, because choosing k of M requires all M, and M is known only at that point. Selecting by arrival order would instead make the retained set depend on the schedule. On the same ground the cap ranges over a state's own matches only and not over matches forwarded to it from an ancestor: a forwarded match races the completion, so a cap that counted them would retain a different set at a different worker count. Forwarded matches are instead thinned individually, on the same terms as discovered ones, at the point they arrive—without which the sampled subgraph would depend on whether forwarding was enabled.

Finally, the per-state and per-transition retention controls are one knob under full capture and two only under quotient exploration. Under full capture raw states stand in bijection with the events that create them, so one coin per new state and one coin per transition are the same coin; under quotient exploration a canonical state has many incoming transitions and the two separate.

Sampling and synchronization. Sampling constrains parallelism through the order in which decisions become final. A sample cannot be acted on before its population is complete: a rewrite cannot be undone, so a match that has been applied cannot later be evicted by a match that arrives afterwards. Any reservoir must therefore be finalised before its members are rewritten, and finalising requires knowing that the stream of candidates has ended. A scheme that samples from all matches produced in an evolution step consequently needs every worker to finish that step before any rewriting begins—a global barrier, in an engine that otherwise has none.

The question is therefore not whether to have a barrier but how local the completion condition can be made. The engine requires a sampling population to be one that *completes locally*: its completion must not depend on knowing anything about work outside the object it is keyed on. A population keyed on a single (state, rule) pair satisfies this, because the engine can tell when that pair has no matches outstanding without consulting any other state; a population keyed on an evolution step does not.

Restricting to locally-completing populations removes less than it appears to, because the global population was not delivering a uniform sample either. Under the step-wide scheme each match job received a reservoir of the same capacity regardless of how many matches it saw, so a pair producing a thousand matches contributed no more samples than one producing twenty; the pooled sample was already weighted toward sparse strata before any shuffle, and shuffling a skewed pool does not remove the skew. Uniformity over the union of all matches was not a property that scheme had. Uniformity within a (state, rule) stratum was, and is the property its test checks. Keying the population on the state therefore preserves what was true and removes the barrier.

5.13 Sessions

Evolution is also exposed as a session rather than only as a single call. A session is opened on a rule set and initial condition, advanced by a requested number of steps, queried for any of the derived structures, and closed. The unexplored frontier is retained between steps, so advancing a session continues the existing exploration instead of recomputing it from the initial state, and a caller may inspect intermediate results before deciding how much further to evolve.

The frontier is a separate structure from the identity map, and it has to be: one bit cannot answer both questions a continuation asks. The deduplication map records that a state has been seen, and a worker reads that same answer as meaning the state has been expanded. Carrying only the map across calls therefore fails in one of two directions depending on who owns it, and both were measured on the same workload—a two-edge to three-edge rule advanced to depth two and then to depth three, against seven states and six events for a single run to depth three. Rebuilding the maps per call gave ten states and nine events: nothing was recognised, so the second call re-derived as new every state it already had. Giving the caller ownership of the maps gave five states and four events: everything was recognised, so nothing was re-expanded and the new depth was never reached. A session therefore carries both the maps and a frontier—the states whose expansion was refused by the step budget rather than by deduplication—which is the distinction the two failures are between. The device implements the same separation as the host, since a continuation that behaved differently on the two targets would not be the same session.

Every session reply reports the frontier, because a caller cannot steer a continuation toward states it cannot name, and a step may name a subset of it: the named states are expanded and

every other frontier entry is put back, so steering narrows what runs next, never what remains reachable. Naming a state that is not on the frontier is an error rather than an empty step—a caller steering toward a state the exploration has already passed would otherwise get a silent no-op indistinguishable from a branch with no successors. Retention is also why each frontier entry carries its own depth: after a steered step, entries stranded by different budgets sit on the frontier together, and an entry resumed at a uniform depth would have its subtree truncated by steps it never took. Both devices serve this contract, resolving the selection against the frontier as it was last reported.

5.14 Event Canonicalization

Events can be canonicalized at multiple granularities:

ByEndpointStates: Two applications are the same event when they run between the same pair of canonical states. $\mathcal{O}(1)$ signature computation.

ByConsumedProducedEdges: The signature additionally carries the canonical ranks of the edges the application consumed and produced. $\mathcal{O}(E)$ signature computation.

DistinctApplications: No signature is computed and every application is its own event. This is the identity the raw causal graph is defined over.

The three refine one another in that order, so the canonical event count is non-decreasing across them; Table 18 reports it per workload. ByConsumedProducedEdges requires the edge correspondence between isomorphic states, which is the per-edge canonical rank produced by the same individualization–refinement pass as the state's hash.

Event identity does not inherit the state-identity mode. Event identity is defined over isomorphism classes of states, and the specification makes it independent of which state-identity mode a run selects. Resolving it through whichever hash the state mode happens to use would break that independence. The Weisfeiler–Leman hash is isomorphism-invariant but strictly coarser than the exact form, so outside the exact state mode more states share a representative, the edge correspondence between an event's endpoints resolves to a coarser one, and the event identity derived from it coarsens with it. The consequence is measurable rather than theoretical: on the binary-growth corpus case the same run reports eight events under the exact state mode and six under the approximate one, a difference produced entirely by the state mode leaking into a quantity that is not supposed to depend on it.

The engine therefore resolves event representatives through the exact invariant in *every* state mode whenever event canonicalization is enabled, independently of the hash the state mode uses for state identity. The two are separate values: one decides state identity, the other resolves event representatives. The exact hash is computed only when event canonicalization requires it, so a run with event canonicalization disabled does not compute it and behaves as its state mode specifies. When it is enabled, the same individualization–refinement pass yields both the exact hash the event path resolves through and the per-edge ranks that identify consumed and produced edges, so the guarantee costs one pass per state rather than two per event.

Configuration dependencies are reported, not absorbed. The canonical-class event identity and quotient exploration are both defined over canonical states and their edge orbits, and those orbit tables are produced only by the exact state-canonicalization mode. A run that asks for the first while selecting a weaker state mode is therefore asking for something the configuration does not supply. The engine neither raises the state mode behind the caller's back nor proceeds as though the request had been met: it builds the causal graph by the raw-edge rendezvous, which is well defined without orbits, and returns a warning naming the dependency

and the setting that satisfies it. The general form of the rule is that a request the configuration cannot support produces a stated substitution rather than a silent one, on the grounds that a result quietly computed under a different definition than the one requested is indistinguishable from the requested one at the call site.

6 GPU Acceleration

Multiway evolution is a poor fit for the shape of work a GPU is usually given. The frontier is irregular and data-dependent, its width is not known before the step that produces it, and each state must be canonicalized—a refinement loop with data-dependent control flow—before it can be compared with any other. A per-phase kernel launch would put a host round trip between every one of those stages. The design here is therefore a single resident megakernel that keeps the frontier on the device across steps, and the subsections describe what that costs: how canonicalization and matching are expressed without host control flow, how work is queued and termination detected among resident blocks, and how memory is managed when no allocator is available.

6.1 Megakernel Architecture

Traditional GPU algorithms launch separate kernels for each operation, incurring a host round trip between every stage. That cost is fixed per launch and does not shrink with the problem, so it dominates exactly where multiway evolution spends most of its steps: the early frontiers, which are small and numerous. Our *megakernel* approach instead runs a single persistent kernel for the whole evolution, keeping the frontier, the match queues and the epoch counter resident on the device between steps, so a step costs a device-side barrier rather than a launch. Figure 4 shows the arrangement: the queues and the shell that services them, with expanded states re-entering as match tasks without returning to the host. What the design buys and where it stops paying are measured in Section 8—it is 16.5× a per-step `evolve()` at depth four and indistinguishable from it by depth eight, once the evolution itself dominates the constant being amortized.

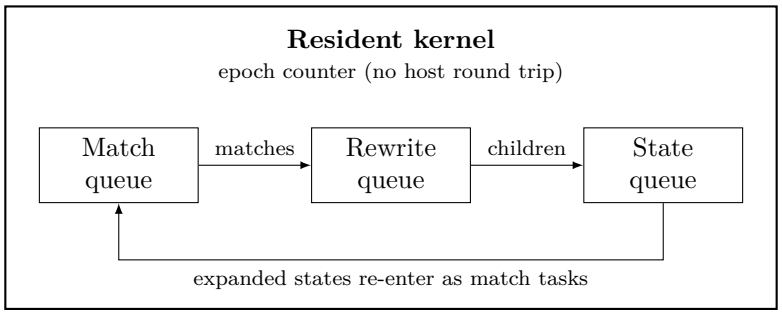

Figure 4: Queue structure of the resident device kernel. Work circulates between the three queues without returning to the host; an epoch counter separates evolution steps.

6.2 The Constraint the Design Follows From

The resident kernel exists to remove a host round trip per evolution step. That aim is only met if nothing crosses the host–device boundary during evolution at all: rules and the initial state are uploaded and the queues seeded before the launch, results are read after it, and in between the device decides what work exists, which worker takes it, and when the evolution has finished. A round trip introduced for any other purpose has the same cost as the per-step round trip the design removes, so the constraint admits no exceptions.

Two properties of the device path follow from it rather than being chosen independently.

Capacity overflow cannot be resolved by retrying with a larger allocation. Enlarging a pool means allocating on the host, which is a round trip. A device-resident loop must therefore record the overflow and return the work completed so far, which is the behavior described in Section 5.7. On the device that behavior is not one option among several; it is the only one available, and it is why capacities are sized from configuration before the launch.

The canonicalizer's scratch cannot be sized by measuring the workload. Sizing a scratch slot to the largest state in a batch requires knowing the batch, and a resident loop has no batch: states arrive continuously and the largest is not known until the run ends. One resolution is a slot pre-sized from a configured maximum with a coarser hash used above it, which would reintroduce the possibility of two states receiving the same key because one was too large to canonicalize exactly—a silent loss of the exactness property of Section 3, bounded by a configuration value rather than eliminated. The resolution taken instead is a device-side arena from which each state claims a slot sized from its own vertex, edge and occurrence counts. There is consequently no approximate fallback in the device's exact path: exhausting the arena is a capacity overflow, recorded and reported like any other, and never a quietly coarser identity.

6.3 GPU Canonicalization

The device does not reimplement canonicalization. The individualization–refinement body is a single source compiled for both host and device, so host and device compute the same canonical hash for a given state by construction rather than by two implementations being kept in step. What is device-specific is the orchestration only: how a state is flattened out of the adjacency representation, and where the core's scratch comes from.

The parallelism is across states, not within one. A worker canonicalizes one state per call, running the shared body serially, which suits a workload whose states arrive continuously and number in the thousands rather than one whose individual instances are large. The scratch for that call is sized from the state's own edge and occurrence counts and claimed from the device arena of Section 6.2, so there is no per-state ceiling and no batch to measure.

There is consequently no approximate fallback anywhere on this path. A state the exact path cannot key—because the arena is exhausted, or because the search wants more individualization depth than the device attempts—is reported as a capacity overflow and the run is retried with a larger reservation. It is never keyed by something coarser. This is not a tuning preference, because isomorphism-invariance fails in only one direction: a 1-dimensional Weisfeiler–Leman hash never separates two isomorphic states, but it does merge non-isomorphic ones, which the engine demonstrates constructively on the prism graph against $K_{3,3}$ at six vertices and on the rook's 4×4 graph against the Shrikhande graph. Nothing bounds how often an evolution reaches such a state, so no collision rate measured over some other corpus would license treating the merge as rare. The only additional device-side hash is the content-ordered key that the AUTOMATIC identity requires, which applies the same rule as the host.

Where a device-side event signature must stand in a raw edge identifier because an edge's canonical rank was unavailable at the moment the event was stamped, the substitution is counted. Such a signature is not an isomorphism invariant, and the count is what lets a caller comparing event totals across devices see that it occurred rather than infer a disagreement in the evolution.

6.4 GPU Pattern Matching

The unit of matching work on the device is a (state, rule) pair, handled by one block whose threads stripe the candidates for the first pattern edge. Parallelism across states and rules is therefore the outer axis, with the inner axis inside a single join.

The device keeps two indices over the edge pool, a signature index and a vertex inverted index, each filtered by the state's edge set at query time, and divides the work between them:

the signature index seeds the first pattern edge; every subsequent edge is driven by the vertex inverted index through a *pivot variable*—a left-hand-side variable that is guaranteed to be bound by the time that edge runs, which the connectivity ordering of the pattern edges establishes. Looking the pivot's binding up in the inverted index yields a candidate list bounded by that vertex's degree, typically a few entries, in place of a signature bucket that holds thousands on a dense state. This is the division adapted from HGMatch [12]: signature index for the seed, inverted index for the extensions. The host answers the same two questions from the state's ancestry (Section 4.2).

One detail of the semantics is computed on the host so that the device does not recompute it per match. Because distinct pattern variables may bind to the same vertex, a single pattern signature is compatible not with one data signature but with every coarsening of its partition. The list of compatible signature hashes for each pattern edge is therefore computed once on the host when the rule is added, and the kernel seeds from the index directly rather than enumerating coarsenings while matching.

6.5 Device Work Queues and Termination

The queues of Figure 4 are bounded lock-free multi-producer, multi-consumer ring buffers in device memory. Workers are producers and consumers of the same queues—a rewrite produces states, which become match tasks—so a single-producer structure would not serve, and a bounded one is required because device memory is reserved in advance.

Each slot carries a sequence number, and that number alone determines whose turn the slot is: it equals the position a producer is reserving when the slot is free, one more than that position when the slot holds an item for the consumer reserving it, and less than the position when the queue is full for a producer or empty for a consumer. Slot i starts at i, a producer publishes with the position plus one, and a consumer releases with the position plus the capacity, so a slot advances by exactly one capacity per lap and the three tests remain exact across wraps. The ordering is carried by that handshake—a release store by the producer synchronizing with the consumer's acquire load—so the head and tail cursors, which only allocate positions, are relaxed.

Reservation is a compare-and-swap on the cursor rather than an unconditional increment. The difference matters precisely because the same workers both produce and consume: an unconditional bump has nothing to undo with, and a rollback or a give-the-slot-back store can land on a slot another thread has already taken, which either loses an item or hands one out twice. In a queue whose producers are its consumers, a lost item is not a dropped unit of work but a run that never finishes, because the termination detector waits for a completion that can no longer occur.

A resident kernel whose workers pull from queues cannot end when a launch ends, so termination is detected rather than assumed. Each queue role maintains two monotonic counters, one for work pushed and one for work drained, incremented with release ordering so that any payload the work touched is visible to whoever observes the counter. A detector reads them with acquire ordering and looks for every pair to be equal. Equality alone is not sufficient: it occurs mid-evolution whenever every in-flight task happens to have just finished without yet emitting its follow-ups. Exit is therefore signalled only when equality holds across two consecutive observations separated by a backoff interval, so that nothing was pushed during the window between them. Workers test the exit flag at the top of their outer loop, and read it with acquire ordering so a worker that sees it also sees everything the detector saw before setting it.

Detecting termination incorrectly truncates the evolution without an error, so the device result is compared against the host's on every corpus workload rather than only checked for the absence of reported errors.

6.6 Memory Management

Device memory is reserved before the kernel launches, since a resident kernel cannot call the host allocator. Edge and state pools are pre-allocated with atomic counters, and scratch whose size is known only once a state's dimensions are seen—the canonicalizer's working arrays, principally—is carved from a device-owned bump arena.

The arena's size is determined by the grid rather than by the workload. A persistent kernel's blocks do not retire and get replaced—they live for the whole evolution—so the grid is the worker count, and each resident worker holds its own arena slot at a time. Arena demand therefore scales with the grid, and sizing it from the state budget alone starves it as soon as the grid grows. The arena is one shared bump pool rather than a partition per worker, because a worker keeps one slot and grows it to the largest state it personally canonicalizes: what has to be provided is the average share across workers, not a per-worker maximum. The default grid is eight blocks per streaming multiprocessor. There is no free operation: a worker finishes with its scratch before taking its next item, so a slot is reused and re-claimed only when a larger one is needed, which with doubling wastes at most one slot per block, bounded by that block's peak. The whole arena is reset at the end of the run.

Where a pre-sized region proves too small for a state, the overflow is reported through a device-side error channel and the work is retried with a larger reservation, which is the mechanism behind the capacity behavior of Section 5.7.

7 Verification of the Concurrent Surface

The engine holds no lock, so every property that a lock would have enforced is instead a consequence of an ordering argument. Arguments of that kind are checked poorly by testing. A stress test samples interleavings, so a clean run bounds nothing: on this engine a determinism gate of 150 runs – 90 recorded, then 40 and 20 more under load—explores a vanishing fraction of the schedules its own thread count admits, and the schedules that matter are not the likely ones. Two verification layers are therefore maintained alongside the test suite, and they are bounded in different directions.

Bounded programs, all executions. The first layer uses GenMC [28], which enumerates the executions of a bounded concurrent program under the RC11 memory model [29]. For the program it is given—that thread count, that operation count, those inputs—it explores every interleaving and every reads-from choice the model permits, so a clean run is a statement about all executions within the bound rather than about the ones that happened to be scheduled.

Three harnesses carry a weaker guarantee than that, and each says so in place. Two of them bound the number of context switches, which the checker permits only under sequential consistency, so those two establish their property for the sequentially consistent executions of their program rather than for every RC11 execution of it. A third samples executions instead of enumerating them, because its shape cannot be exhausted at the size that exhibits the behaviour it is about; a clean sample is evidence rather than proof, and the same property is proved by enumeration at a smaller shape by a separate harness. The remaining harnesses are exhaustive over RC11 within their stated bound.

Each harness is a small program that includes the engine's own header and calls its own functions, rather than a hand-written model of the algorithm. The constraint is deliberate: a model drifts from the code as soon as the code changes, and a re-implementation establishes a property of the re-implementation. A harness that includes the header stops compiling when the header changes shape, which couples it to the code it is about. Two harnesses cannot meet the constraint, because the protocols they check live inside functions the interpreter cannot take, and both say so in place: their transcriptions are copied verbatim and run over the real

underlying structure.

The properties checked are the ones whose failure would be silent. For the concurrent map: that get-or-create admits exactly one winner and returns that winner to every caller, so a *was_inserted* result means what its callers assume. That this survives a table resize running concurrently with the inserts that triggered it, and survives two successive growths. And that the flag comes from the publishing exchange rather than from a value comparison, so repeated offers of the same value for one key do not each report success. For the deque: that the owner popping the back and a thief popping the front never extract the same item, checked at the single configuration where the two ends address the same slot. For the match rendezvous: that a match is claimed exactly once and never dropped on a collision, and, for the rendezvous between an instance and the matches captured for its class, that every pair meets. For the per-instance application list: that of any two applications linked into one list exactly one observes the other, permitting the branchial pair they induce to be emitted once with nothing to deduplicate it against (Section 5.10), and that a walk from a node enumerates every node linked before it. For the per-worker scratch arena: that no two workers are handed the same index. For the key set: that two threads inserting one key while a third insertion forces a growth beneath them produce exactly one reported insertion, and that enumeration emits each key once across that growth—two properties checked by separate harnesses, because they are independent and a single harness that bundled them would not say which one a violation belonged to. A third property class covers *readers* racing a growth: a membership query for a key that settled before the query began must answer present in every execution, even mid-carry, when the key's slot in the retiring table is already sealed and its copy has just landed in the successor. The chain walk is oldest-first for that reason: a newest-first walk visits each table on the wrong side of the carry hand-off and can report absent for a key that is present, and the twenty-execution counterexample distinguishing the two orders is in the harness. The same read-order condition governs the map's drained-table skip, where the calibrated break is reading a table's skip flag only when the walk arrives at it. For the scheduler: that a worker which parks is always woken, which is a liveness property and is checked as one—removing either of the two sequentially consistent fences in the submit/park protocol makes the checker report a non-terminating spin loop.

The checker itself. Running an engine of this size through GenMC required changes to the checker, all of which are shipped with the engine as a patch against release 0.17.0. Four are correctness fixes: `operator new[]` is intercepted as an allocation, as `operator new` already was; a `memcpy` is promoted over the element type its destination names, with byte stores for a tail shorter than the last element and alignment-width stores for an opaque `memset`; the read half of a failed compare-exchange carries the failure ordering rather than the success ordering; and no liveness violation is reported while a thread is blocked by an assume, a helped compare-exchange or a confirmation (upstream pull request 58). One is a limitation in the lowering of copies whose operands the promotion pass cannot type—a call result, or a load from a lambda's closure field—which were lowered as loops at the intrinsic's alignment, so that a program's two-byte field read fell inside a four-byte store, an access the checker's value resolution does not answer and reported as uninitialised memory; the promotion now runs once before and once after scalar replacement and register promotion, when such operands have become the allocas they held, and the report names a read that lies inside a wider write. The rest are instruments: a progress heartbeat over the transformation passes and the exploration, and, in estimation mode, an attribution of the state-space estimate to the reads and writes that produce it—every access with more than one alternative is booked against its source line with weight $\log_2$ of the alternative count, and the booked total equals $\log_2$ of the estimator's sample per execution. That attribution is what identified the two changes that made the composed harnesses tractable: an idle worker's empty pop of its deque wrote its `bottom` word twice per look, giving every

thief's read of that word one reads-from alternative per store, and the workers were spawned with distinct arguments, which kept the checker's symmetry reduction from applying.

Unbounded worker count, a transcribed model. The second layer uses TLA+ and its model checker [30] on the match forwarding protocol, which is the property whose silent failure is most costly: a lost match removes the entire subtree below it while leaving the run internally consistent. Every in-flight protocol step is held in a bag from which any enabled step may fire, so the interleavings subsume every worker count rather than a bound on it; what is bounded instead is the state count. The trade is stated where the specification is: this layer is a transcription, so a change to the code does not change the model, and each specification header names the code it transcribes.

The specification is checked in four configurations, differing in whether the claim winner owns both the store and the propagation and whether submission is batched (Table 3). The shipped configuration is forwarding-complete at the model bound. The configuration without the ownership invariant and without batching violates the property. Its counterexample is the shape the invariant exists to exclude. On a chain $s_0 \to s_1 \to s_2$, a match is discovered at the root after s_2's pull has run; s_1's pull wins the claim and stores without propagating; the root's push then finds the claim taken and skips its recursion. That leaves s_2 quiescent with a match that is valid for it and not stored. That the model reaches the failure it exists to exclude is the calibration of the model. The remaining configuration shows that batching alone masks the broken pull at this bound, which is why the ownership invariant is not treated as redundant with it.

Table 3: **Forwarding completeness under TLA+.** Four configurations of the same specification, exhaustive at the model bound of four states, three matches and three edges, with a mid-chain discovery enabling a grandchild. *Ownership* is the claim winner owning both the store and the propagation; *batched* is per-state submission after matching completes.

Ownership	Batched	Verdict	Distinct states
yes	yes	complete	117,005
no	yes	complete	85,777
yes	no	complete	479,005
no	no	**violated**	trace depth 13

A second structure in the same layer. The segmented array carries the states, the events, the edges and the per-instance application lists, and its ordering invariant is that if a segment exists then so does every segment below it—a reader that finds segment k may index any position beneath it without further checking. That invariant is specified rather than tested, because the structure allocates segments on demand from concurrent appenders and the failure it guards against is a read of an unallocated segment rather than a wrong value. The shipped protocol checks clean at 2,284 distinct states and a complete-graph depth of 21; the configuration that creates only the segment asked for violates *DenseBelowCount*. The calibration also fixes the bound: at two appenders the broken protocol is *also* clean, since one claim each means the index that would expose the gap cannot be reached until a holder of the lower segment has published, so three are required. It is specified here rather than checked with the tools of the previous layer because that checker does not survive constructing this structure—a limitation of the tool, recorded where it applies rather than presented as a choice.

What a sampled execution reports. The two layers above bound the schedule; the test suite does not, and no sanitizer changes that. A sanitizer changes what a given execution is

able to report, which is the second half of the objection to testing: a stress run samples few schedules, and the schedules it does sample can still pass while holding a race, an out-of-bounds access, or a read of memory nothing wrote. The suite is therefore run under ThreadSanitizer, AddressSanitizer and UndefinedBehaviorSanitizer, and the device suites under the CUDA memory and initialization checkers; all five report nothing on the shipped tree, and all five fail the build on a finding. The initialization checker detects a case the others do not: a host readback of device memory that no kernel wrote returns whatever the allocator last left at that address, so an assertion over it succeeds or fails according to allocation history rather than according to the code, and neither a test nor a model checker over the concurrent structures observes it.

8 Experimental Evaluation

The evaluation answers five questions, and each table below is labelled with the one it addresses.

Q1 (correctness). Is the state identity exact—does the engine identify two states if and only if they are isomorphic—on every workload in the corpus, checked against an independent oracle? (Table 4)

Q2 (cost against the reference). How does evolution time compare with the Wolfram implementation computing the same object, and how does that ratio behave with depth? (Table 6, Figure 5)

Q3 (parallel scaling). How far does evolution scale with worker count, does that limit depend on the rule, and what resource sets it? (Tables 10, 11, 13; Figure 7)

Q4 (device). Where is the GPU faster than the CPU, and at what grid size does the device saturate? (Tables 8, 14)

Q5 (attribution). Which contributions account for the performance, and does any of them change the computed answer rather than its speed? (Tables 16, 7, 18)

Every table below is written by `tools/dev/paper_tables.py` from the instrument named in its caption, and records the commit and machine it was produced on. The one exception is Table 15, transcribed from an Nsight Compute session because its reports are not text; its caption says so. Captions carry a **T***n* identifier alongside the float number: **T***n* names the generated fragment (`paper/tables/t`*n*`_*.tex`) and so ties a table to the tool that wrote it, while the float number is LaTeX's. The two do not correspond, and the **T** sequence has gaps where a fragment was retired. Numbers are medians over repeated runs in one session; a comparison between two arms is always taken in the same session, because this machine's run-to-run drift exceeds most of the effects being reported.

8.1 Experimental Setup

Hardware: AMD EPYC 9174F (16 cores, 32 threads, one socket), 503 GB RAM, NVIDIA RTX 4090 (24 GB, compute capability 8.9), on bare metal. The cores are homogeneous, so a speedup column divides by a comparable quantity: worker threads are pinned to one hardware thread per physical core, and each table's provenance line names the CPU set it was pinned to, the machine, and the load at the moment the measurement started. A table moved to another machine carries that machine with it. The Wolfram comparison of Table 6 is the only measurement not on this host: it needs a licensed kernel, and its provenance line names the machine it was taken on. The remaining tables and figures were not all produced by one invocation and their provenance lines name several commits, but that spread is bookkeeping rather than a difference in what was measured: no file under

the engine's own source directories differs between any of those commits and the one this document was built from. Table 6 and Figure 5 are the exception in this sense too—they were taken at an earlier commit, across which 114 engine files have since changed—so they are the one measurement here that describes an older engine than the rest.

Software: Ubuntu 24.04, Linux 6.8, GCC with C++20, CUDA 12.0, Wolfram Language 14.3.

Workloads: The rule-type corpus of 17 workloads from `reference/oracle_corpus.hpp` (single/multi rule; productive, idempotent, reductive, and mixed-arity rules; several initial-state shapes), plus larger depths of the canonical Wolfram rule $\{\{x, y, z\}\} \to \{\{x, y, w\}, \{y, z, w\}\}$ for the scaling and GPU curves.

Methodology: Wall-clock figures are medians over repeated runs on an otherwise idle machine, with the workers pinned to a named homogeneous core set; a run taken while the machine was loaded is marked in the table's provenance line rather than published as though it were quiet. Counts, instruction-level measurements and `getrusage` figures are deterministic and are reported from a single run. One exception is stated where it applies: the left-hand-side sweep of Figure 8 reports the *minimum* of its repeats rather than the median, because contention can only make a run slower, so the minimum is the estimate interference cannot inflate—and that sweep admits a timed run only after checking the machine is quiet.

8.2 Correctness and Memory (Tables T1 and T3)

Correctness comes first because a performance number for a wrong answer is not a result. Every workload in the corpus is evolved and its canonical state set, event set, causal relation and branchial relation are compared against the independent reference implementation; all seventeen agree exactly. Table 5 then reports what those runs cost the global allocator, which is the claim the arena exists to support: allocation count is set-up performed once, so it does not grow with the work.

Table 4: **T1 — Exactness and deterministic memory.** One row per corpus workload: canonical states, events, causal edges, branchial edges, arena bytes, heap bytes and heap allocations, all in FULL (exact) mode, single-threaded. Each workload is evolved to its own depth, the `oracle_steps` recorded with it in `reference/oracle_corpus.hpp`, chosen so the brute-force cross-check stays tractable; counts are therefore comparable down a column and not across rows. The *Exactness* column must read EXACT for every row, checked against the brute-force isomorphism oracle. Deterministic memory is measured as arena `bytes_allocated()`, not RSS, for reproducibility. Source tool: `tools/cost_matrix.cpp`.

Workload	Class	Canon. states	Events	Causal	Branchial	Arena B	Heap B	Heap allocs	Exactness
binary-growth	productive/arity2	36	873	872	0	1325896	3118060	64	exact
wolfram-2to4	productive/arity2	45	126	184	111	624224	1500540	58	exact
triangle	productive/mixed-conn	36	124	192	249	615024	1517028	59	exact
reductive-2to1	reductive/arity2	4	15	12	5	500184	1499068	57	exact
idempotent-2to2	idempotent/arity2	1	6	10	0	485312	1499020	56	exact
self-loop	self-loop/arity2	2	1	0	0	356608	1253308	55	exact
mixed-arity	mixed-arity	7	6	5	0	417784	1253324	55	exact
arity3-growth	arity3	65	153	152	0	519320	1500596	57	exact
disconnected-lhs	disconnected	1	126	248	63	603240	1499314	58	exact
arity4-growth	arity4	23	33	32	0	437616	1498916	55	exact
mixed-arity-lhs	mixed-arity/join	2	1	0	0	373552	1253284	56	exact
mixed-arity-init	mixed-arity/init	4	4	0	0	391528	1253316	57	exact
arity3-to-2	arity-reductive	4	4	0	0	391636	1253348	56	exact
arity1-with-binary	arity1/mixed	2	1	0	0	373568	1253268	56	exact
cycle4-automorphic	automorphism	7	68184	109992	68184	168694332	187655636	218	exact
star4-automorphic	automorphism	201	68184	56360	0	92987304	109025524	171	exact
multi-rule	multi-rule	36	237	214	108	674352	1537596	59	exact

Table 5: **T3 — Global allocator use during evolution.** One row per corpus workload, ordered by work performed. `cost_matrix` counts every call to global `operator new` during the measured evolution. Allocations per event fall from 56 on the smallest runs to 0.0025 on runs four orders of magnitude larger: the count is fixed set-up performed once per run, so it is divided by an event count that grows while it does not, and per-event allocation is served from the arena rather than the allocator. No before/after comparison is given for the contributions that produced this figure. Each replaced path was deleted in the commit that replaced it, the work spans dozens of commits, and the instrument that measures arena bytes did not exist at the start of that sequence; two numbers from two different instruments would not constitute a comparison. Source: `tools/cost_matrix.cpp`.

Workload	Canon. states	Events	Arena B	Heap allocs	Allocs per event
self-loop	2	1	356608	55	55.0000
mixed-arity-lhs	2	1	373552	56	56.0000
arity1-with-binary	2	1	373568	56	56.0000
mixed-arity-init	4	4	391528	57	14.2500
arity3-to-2	4	4	391636	56	14.0000
idempotent-2to2	1	6	485312	56	9.3333
mixed-arity	7	6	417784	55	9.1667
reductive-2to1	4	15	500184	57	3.8000
arity4-growth	23	33	437616	55	1.6667
triangle	36	124	615024	59	0.4758
wolfram-2to4	45	126	624224	58	0.4603
disconnected-lhs	1	126	603240	58	0.4603
arity3-growth	65	153	519320	57	0.3725
multi-rule	36	237	674352	59	0.2489
binary-growth	36	873	1325896	64	0.0733
cycle4-automorphic	7	68184	168694332	218	0.0032
star4-automorphic	201	68184	92987304	171	0.0025

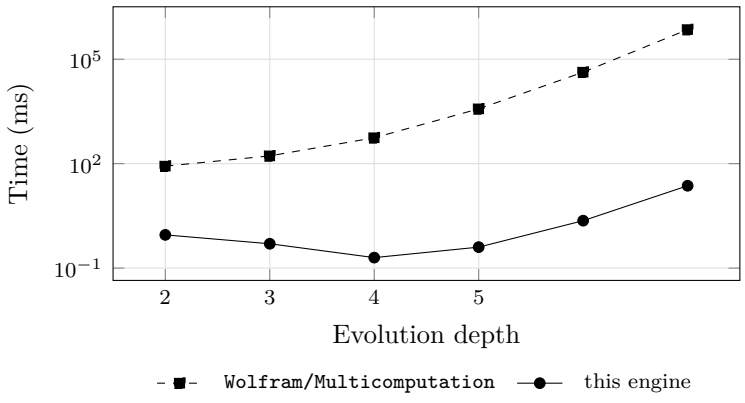

Figure 5: Evolution time against depth for the two implementations, logarithmic in time, on the same rule and initial condition and producing identical state and causal-edge counts at every depth. The vertical separation is the speedup reported in Table 6: 2,770× at depth 4, rising to 30,627× at depth 7. The depth-2 point includes one-time initialization on the Wolfram side, and this engine's times at depths 3–5 are near the timer's resolution.

8.3 Wall-Time Speedup (Table T2)

The comparison is against the `Wolfram/Multicomputation` `MultiwaySystem`, the published implementation this engine reproduces, and against `MultiwayReference`, the brute-force Wolfram-language evolver in `reference/`. All three compute the same state and event counts at every depth, without which the ratio would not be a comparison. Over depths four to seven the engine is between 2,770× and 30,627× faster than `MultiwaySystem`, and the ratio rises monotonically across those four depths. The range starts at depth four because the two shallower rows are not measurements of the same thing: depth two carries the Wolfram side's one-time initialization, and at depths three to five the engine's own time is 0.2 to 0.5 ms, close enough to the timer's resolution that those ratios are lower bounds on the gap rather than precise factors. At depth seven neither caveat applies: the engine takes 23.2 ms against 710 s, so that row is a direct measurement on both sides.

Table 6: **T2 — Comparison against `MultiwaySystem` and against this project's own reference implementation.** The workload is the corpus's `wolfram-2to4`: $\{\{1,2\},\{2,3\}\} \to \{\{1,2\},\{2,4\},\{4,3\}\}$ from the initial state $\{\{1,2\},\{2,3\}\}$. All three compute the same evolution and are timed at their respective interfaces: the native engine at its C++ API, the `Wolfram/Multicomputation` paclet's `MultiwaySystem` through its exposed properties, and the reference implementation (`reference/MultiwayReference.wl`) at its function call. `MultiwayReference` is the brute-force Wolfram-language evolver this project validates its engine against; it is included because `MultiwaySystem` gives the running time of the implementation a user would otherwise run, while `MultiwayReference` gives the running time of a direct, unoptimised implementation of the same definition. Unlike `MultiwaySystem` it returns data rather than `Graph` objects, so its time is not charged for graph construction and the ratio against the engine's C++ core is not an artifact of it. The *Counts agree* column requires all three to have produced the same state and causal-edge counts at that depth; a timing comparison between different state spaces is not a comparison. Every property that implementation exposes is graph-valued (`StatesGraph`, `CausalGraph`, `EvolutionGraph`; enumerated by `reference/authority_properties.wls`), so its measured time necessarily includes construction of Wolfram `Graph` objects, and no interface returning the state set alone is available. State and causal-edge counts agree exactly at every depth. The *Engine ms (paclet)* column reports this project's own Wolfram-layer cost and is excluded from every speedup quoted here; at depth five it is 166× the engine's own evolution time (66.3 ms against 0.4 ms), and it is the reason no speed claim in this paper is taken through that layer. The speed this paper reports is the engine's native speed with serialization excluded, which is the only figure comparable across the three implementations: `MultiwaySystem` cannot separate its graph construction from its evolution, and `MultiwayReference` does not build graphs at all. The depth-2 row includes one-time kernel and paclet initialization on the Wolfram side, and the native times at depths 3–5 are near the timer's resolution. Source: `tools/quotient_reconstruction_cost_probe` and `reference/bench_authority.wls`.

Depth	States	Authority ms	Reference ms	Engine ms (paclet)	Engine ms (C++ core)	Speedup vs authority	Speedup vs reference	Counts agree
2	3	84.8	10.6	51.1	0.9	94×	12×	yes
3	4	167.2	36.7	32.6	0.5	334×	73×	yes
4	5	554.0	153.5	42.7	0.2	2770×	768×	yes
5	6	3750.7	835.3	66.3	0.4	9377×	2088×	yes
6	7	42399.9	5589.2	467.8	2.3	18435×	2430×	yes
7	8	710537.0	46944.8	6882.4	23.2	30627×	2023×	yes

8.4 Quotient Exploration (Table T6)

Quotient exploration collapses states related by an automorphism of the rule set into one representative, so the number of states actually expanded grows far more slowly than the raw count. At depth seven the raw evolution reaches 5913 states where the quotient reaches 28

classes, and the reconstruction recovers the raw events, causal edges and branchial pairs from those classes exactly. The saving is in what is expanded, not in what is reported.

Table 7: **T6 — Quotient against full expansion.** Events produced and wall time for quotient exploration—expanding each canonical state once, at its shortest reachable depth—against full expansion, at increasing depth on the Wolfram rule and with the same canonical closure. Both modes produce the same canonical state set; the columns report the difference in events produced and in wall time. The quotient arm expands 28 events at depth seven where full expansion reaches 5,913, and reconstruction recovers all 5,913 exactly from those 28. Reconstruction's own cost is *below the resolution of this measurement*: the `quot.+recon` column lands at or a little under the `quot.` column at most depths (4.8 against 5.5 ms at depth seven), which is run-to-run variation on sub-ten-millisecond timings and not a negative cost. What the table establishes is that reconstruction is exact and that it is cheap enough not to register beside the evolution it reconstructs from; a workload large enough to separate the two would be needed to price it. Source: `tools/quotient_reconstruction_cost_probe`.

Depth	Events (full)	Causal (full)	ms (full)	Events (quot.)	ms (quot.)	Events (quot.+recon)	ms (quot.+recon)	Exact
2	3	4	0.6	3	0.4	3	0.3	exact
3	9	16	0.3	6	0.3	9	0.4	exact
4	33	64	0.3	10	0.4	33	0.3	exact
5	153	304	0.6	15	0.4	153	0.4	exact
6	873	1744	2.9	21	0.9	873	0.9	exact
7	5913	11824	23.3	28	5.1	5913	4.8	exact

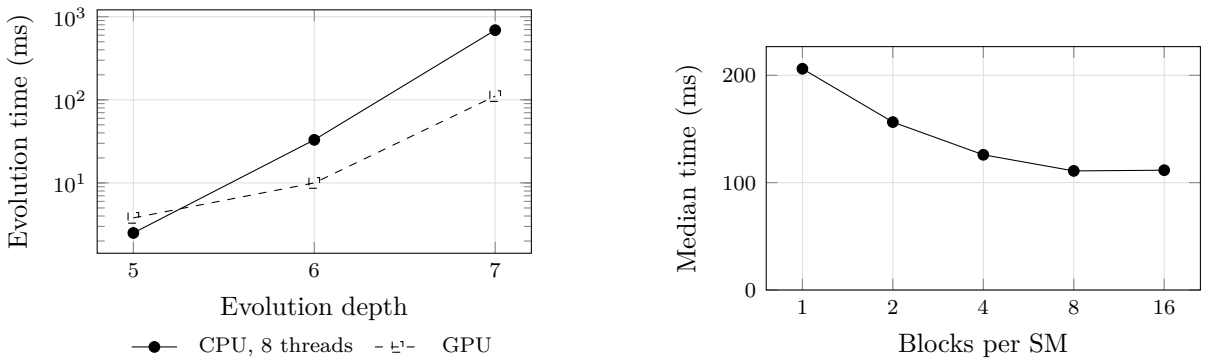

Figure 6: Left: evolution time against depth for the GPU and for eight CPU threads. The device is slower at depth five, where the work per launch does not amortize the launch cost, and faster from depth six. Right: median time against the resident kernel's grid size. Time ceases to improve beyond eight blocks per streaming multiprocessor, which is the shipped default. Both panels plot the values of Tables 8 and 14, which carry the provenance of each measurement.

8.5 GPU (Table T7)

The device is not uniformly faster, and the range over which it is faster has both ends (Figure 6). At depth five the CPU wins, because the work per launch does not amortize the cost of getting it there; from depth six the device is ahead, reaching 6.2× eight CPU threads at depth 7 and more than four times that against a single one.

The resident-kernel design has its own boundary, on the same workload and measured by the same tool (Table 9). Against a conventional per-step `evolve()` the persistent evolver is 16.5× faster at depth four, 13.2× at five, 5.6× at six and 4.2× at seven—and 0.97× at depth eight and 0.98× at nine, where the two are indistinguishable. The advantage is the elimination of a fixed per-call cost, which is visible directly in the `evolve()` column: it is 49–55 ms at depths

four, five and six, almost independent of a state count that grows from 53 to 3867 over that range. Once the evolution itself is large enough to dominate that constant—by depth eight, at 262,144 states—keeping the kernel resident buys nothing. Reporting the speedup without the depth it is measured at would therefore be a statement about small problems.

Table 8: **T7 — GPU against CPU.** End-to-end evolution time for the GPU and for the CPU at one and at eight threads, at increasing depth on the Wolfram rule; branchial edges are constructed through a per-(state, edge) index. Both sides use the same rule, initial condition, canonicalization mode and exploration strategy, and report medians from a single session on the machine recorded in the table's provenance line. At depth five the GPU is slower than eight CPU threads, where the work per launch does not amortize the launch cost; from depth six it is ahead of both columns, and the margin widens with depth. Source: `tools/bench_gpu_evolve` and `tools/bench_cpu_evolve`.

Steps	Raw states	Events	CPU 1t ms	CPU 8t ms	GPU ms	vs. CPU 1t	vs. CPU 8t
5	401	400	4.1	2.5	3.8	1.09	0.66
6	3867	3866	63.0	33.0	10.0	6.27	3.28
7	45317	45316	3200.5	690.1	110.9	28.86	6.22

Table 9: **T16 — The resident kernel against a per-step `evolve()`, by depth.** Both arms are on the device, on the same rule and initial condition, so no CPU arm is involved and the sweep reaches depth nine in minutes. The persistent evolver's advantage is the elimination of a fixed per-call cost, which the `evolve()` column shows directly: about 50 ms at depths four, five and six while the state count grows from 53 to 3,867. Once the evolution itself dominates that constant—by depth eight, at 262,144 states—the two are indistinguishable. A speedup for this design quoted without the depth it was measured at is therefore a statement about small problems. Source: `tools/dev/persistent_depth_table.py` over `tools/bench_gpu_evolve`.

Depth	States	`evolve()` ms	Persistent ms	Speedup
4	53	49.0	3.0	16.55×
5	401	49.9	3.7	13.31×
6	3,867	55.7	10.3	5.43×
7	45,317	468.3	111.3	4.21×
8	262,144	929.4	964.6	0.96×
9	524,288	1695.5	1750.7	0.97×

8.6 Thread Scaling (Table T8)

Strong scaling is reported at three problem sizes rather than one, because a single curve conflates the engine's parallel behavior with the amount of work available to parallelize. The larger problem sustains efficiency far better—0.56 at eight threads against 0.20 at depth five—and the depth-seven curve is superlinear at two threads. The shortfall at high thread counts is the subject of the next two subsections.

8.7 Rule Shape and Where Efficiency Falls (Table T12)

The rule set decides how much parallel work exists, so a scaling curve characterizes the rule as much as the engine. A one-edge left-hand side re-matches by scanning an index and expands each state narrowly; a two-edge left-hand side joins and produces a wider frontier. The two hold efficiency very differently—0.67 against 0.11 at 32 threads—and the one-edge rule's wall time stops improving after 24 threads and rises at 32. Section 8.8 sweeps this axis at four sizes and two further properties of the left-hand side.

Table 10: **T8 — Strong scaling at three problem sizes.** Median wall time and parallel efficiency (speedup divided by thread count) at each thread count, for depths five, six and seven on the same rule. Workers are pinned one per physical core, so the sweep ends at sixteen: this host has sixteen cores and thirty-two hardware threads, and a speedup column that divided by sibling threads would not be dividing by compute. Efficiency, rather than speedup, identifies where added threads cease to contribute. It is higher at every thread count on the larger problem —0.56 at eight threads against 0.20 at depth five—because the fixed serial cost is amortized over more work, and the depth-seven curve is superlinear at two threads (1.04). At sixteen threads the three depths give 0.11, 0.14 and 0.40. Table 13 identifies what the shortfall is spent on. The expanded state set and the (input, output, rule) transition multiset are identical at every thread count and seed. Source: `tools/dev/scaling_sweep.py` over `tools/bench_cpu_evolve`.

Threads	depth 5		depth 6		depth 7	
	ms	eff.	ms	eff.	ms	eff.
1	3.9	1.00	62.3	1.00	3201.6	1.00
2	2.0	1.00	34.8	0.90	1526.3	1.05
4	2.8	0.35	39.6	0.39	989.9	0.81
8	2.3	0.21	31.2	0.25	640.5	0.62
16	2.0	0.12	25.1	0.16	429.5	0.47

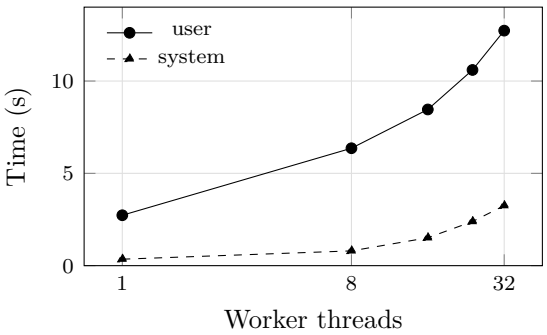

Figure 7: User and system time against worker count, for the runs of Table 13. Across a 32× increase in workers user time rises by 4.7× while system time rises by 9×. System time is the faster-growing term and it tracks the resident set, so what the efficiency decline of Figure 8 is spent on is memory acquisition rather than lock contention.

Table 11: **T12 — Wall time and parallel efficiency by rule shape.** The rule set decides how much parallel work exists, so a scaling curve measured on one rule characterizes that rule. Two shapes are measured at every thread count: a one-edge left-hand side, which re-matches by scanning an index and expands each state narrowly, and a two-edge left-hand side, which performs a join and produces a wider frontier. The one-edge rule reaches 0.43 at eight threads and 0.31 at sixteen, and its wall time stops improving before the sweep ends; the two-edge rule holds well above it from two threads onward—0.95 against 0.68 at two, and 0.67 against 0.11 at 32—as the table's second pair of columns shows. The decline is not thread setup—both serial points are seconds, not milliseconds. This table gives the wall times behind the size axis of Figure 8, which sweeps the same axis at four sizes. Source: `tools/dev/scaling_sweep.py` over `tools/sampling_cost_smoke`.

Threads	one-edge LHS ms	one-edge LHS eff.	two-edge LHS ms	two-edge LHS eff.
1	3035.5	1.00	34072.2	1.00
2	2176.2	0.70	18569.9	0.92
4	1379.7	0.55	9753.2	0.87
8	888.1	0.43	5179.2	0.82
16	616.6	0.31	2238.2	0.95
24	554.8	0.23	1589.5	0.89
32	569.2	0.17	1312.6	0.81

8.8 The Three Properties of a Left-Hand Side

Table 11 varies one property of the rule—how many edges its left-hand side has—and holds two others fixed without naming them. A left-hand side has three independent properties, and each changes the matching problem in a different way:

- **Size**: the number of edges to bind. Each additional edge is another factor in the search, and another constraint that prunes it.
- **Arity**: the number of vertices per edge. Arity changes how much of the state a single edge determines once bound, without changing how many edges must be found.
- **Connectivity**: how the edges share vertices. A path shares one vertex between consecutive edges; a star shares one vertex among all of them; a ring has no endpoint to anchor at; and a left-hand side whose components share *no* vertex cannot be joined at all, so it is matched by enumerating a cartesian product—quadratic in the state size for two components, and worse for more.

Figure 8 sweeps each axis separately, and the result on the first axis runs opposite to the intuition that a larger pattern is harder to parallelize. Parallel efficiency at 32 threads *rises* monotonically with left-hand side size: 0.15, 0.49, 0.67, 0.73 for one, two, three and four edges. The one-edge rule is the worst-scaling shape measured, and it is not scaling badly for want of work: across the four chain sizes of this figure its serial time is the largest. It runs 17.9 s, against 5.7, 10.3 and 12.7 s for two, three and four edges. (Table 11 sweeps two different rules at different depths, so its serial column is not comparable with these.) A one-edge pattern binds one edge per match, so the work between two synchronizations is small and the fixed cost of publishing a state dominates; each further edge adds join work per match without adding a synchronization.

Connectivity moves the curve much further than size does, and in both directions. At 32 threads the ring reaches 1.06 and the branching tree 0.96—superlinear, because a wider frontier lets workers deduplicate states that a serial order would expand—while the path and the hub sit at 0.67 and 0.62. The disconnected shape reaches 0.06: seventeen times worse than the ring on the same machine and the same thread count. That is the widest spread in the whole sweep, and

it is the reason connectivity rather than size is the property to state when a rule set's scaling is being predicted. The disconnected case is reported here rather than in a section of its own, because it is this same axis taken to its limit. Its cost is also not only the cartesian product: on a processor whose cores do not share one last-level cache it is dominated by coherence traffic. Measured on the evaluation machine at a fixed two threads, so scheduling and work distribution are held constant, the same disconnected workload takes 976 ms when both threads share an L3 instance and 1494 ms when they do not, against 1322 ms on one thread—two threads that share a cache speed it up, two that do not are slower than serial.

Edge arity is the weakest axis in one direction and not in the other. At a fixed three-edge left-hand side, efficiency at 32 threads is 0.67 at arity two, 1.015 at arity three and 0.777 for a left-hand side alternating arities two and three. Raising the arity of a path raises the overlap between consecutive edges from one vertex to two, which pins a match's orientation and removes the second way of binding it; the resulting frontier is narrower per state and deduplicates more, which is where the superlinear value comes from.

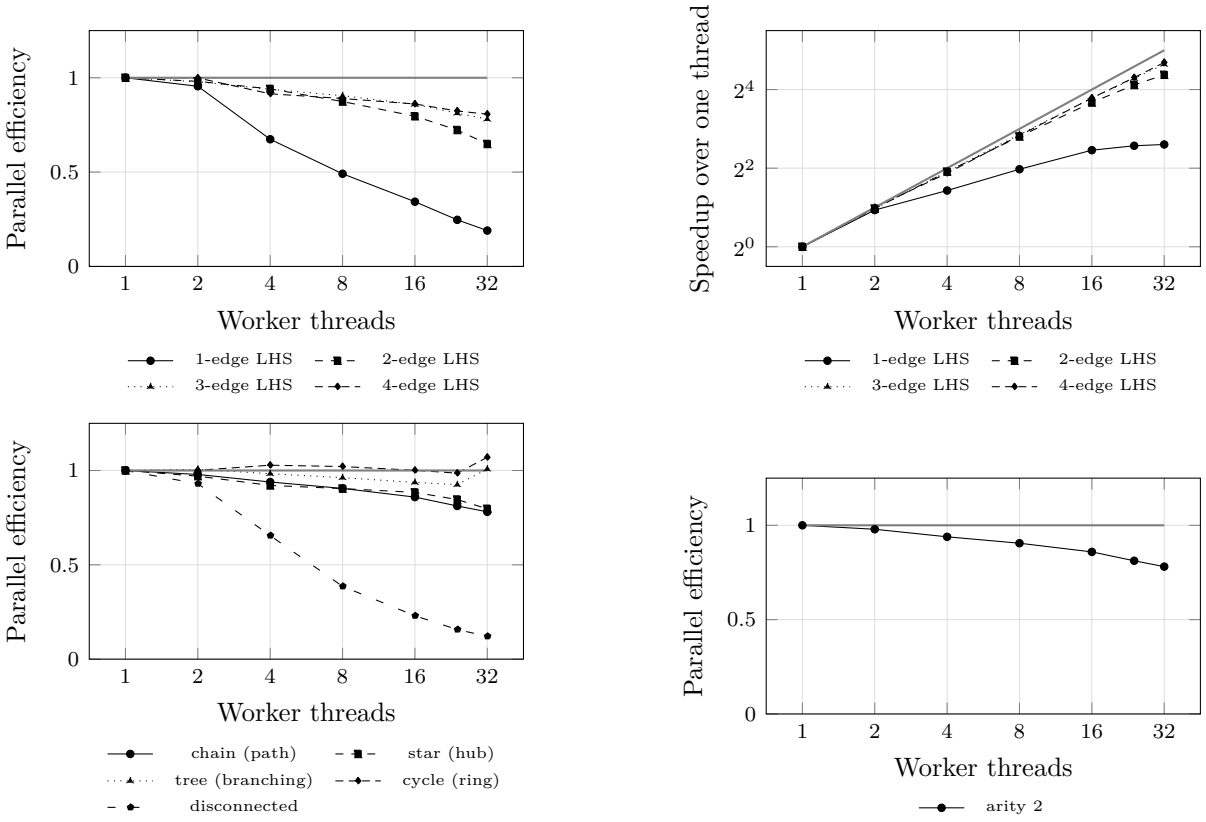

Figure 8: **Parallel efficiency across the three axes of a left-hand side.** Every curve is normalized by its *own* single-thread time, on the same binary, workload and canonicalization mode, so the curves are comparable to each other and none is relative to another system. The grey reference is ideal scaling: $y = 1$ on the efficiency panels, $y = x$ on the speedup panel (upper right, logarithmic on both axes, so ideal scaling is the diagonal). Upper panels: left-hand side size, one to four edges, all binary chains. Lower left: connectivity at comparable size—path, hub, branching tree, ring, and two components sharing no vertex. Lower right: edge arity two and three at a fixed three-edge left-hand side, and one alternating the two. Measured by `tools/dev/rich_sweep.sh`, which admits a timed run only after confirming no other process of this project is running and the one-minute load average is below 1.5; each point is the minimum of its repeats.

8.9 How Far Each Shape Evolves, and How Large Its Relations Become

Wall time is one property of a workload and the sizes of the objects it constructs are another. Two runs can agree on state count and still differ by more than an order of magnitude in the number of branchial edges they build. At around 300k states the two-component shape reaches 3.2M of them while a mixed-arity chain of comparable size builds *none*, and across the sweep the relation ranges from empty to 58.8M—it is over *pairs* of events sharing a parent, so its size is governed by how many matches overlap rather than by how many states exist. Figure 9 plots both, and Table 12 gives the deepest point reached by each shape.

The one-edge left-hand side is the boundary case: it produces *no* branchial edges at any depth, because a single-edge pattern cannot have two overlapping matches at one state to be branchially related. Every shape with two or more edges does.

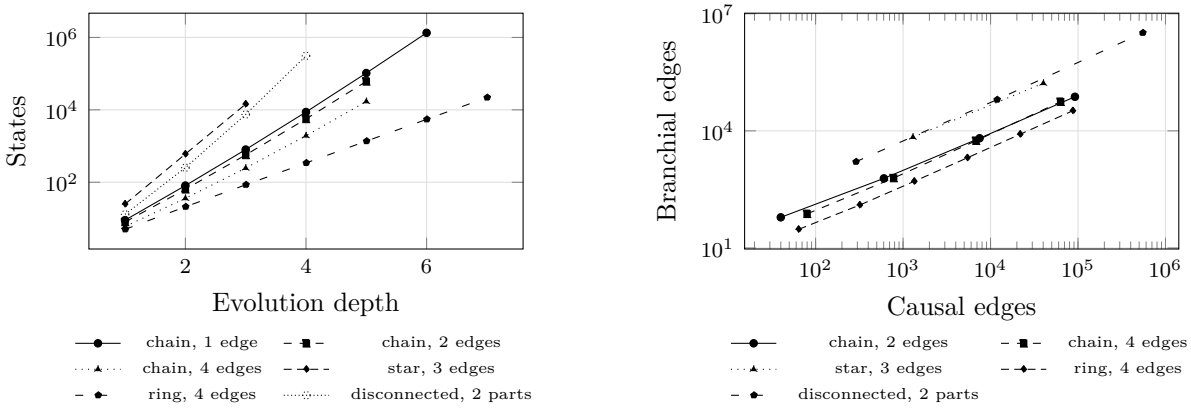

Figure 9: **The state space and the relations over it, by left-hand side shape.** Left: states against evolution depth, logarithmic in states. Each shape is evolved one depth at a time until a single run exceeds the collection budget, so the right-hand end of each curve is a measured boundary for that shape rather than a depth chosen in advance for all of them. **Runs that reached a container ceiling are excluded.** The engine returns a truncated graph with a warning when a container fills, and such a run reports where it filled rather than how large the state space is: measured, one shape totals the same 2,097,149 states whether asked for depth 9, 10 or 11, redistributing that ceiling over however many levels were requested. Of the 192 collected points, 41 are excluded on that basis. Right: branchial edges against causal edges, both logarithmic, every point one (shape, depth) run. A shape's position on this plot is a property of its rule rather than of how long it was run: the shapes that generate many overlapping sibling matches sit high whatever depth they reached. Source: `tools/dev/rich_sweep.sh` over `tools/sampling_cost_smoke`.

8.10 What the Lost Efficiency Is Spent On (Table T13)

A falling efficiency curve does not say which resource is being consumed, and the two candidates imply opposite remedies: contention on shared structures would be fixed by changing those structures, while the cost of memory the run acquires would not. Both are read from `getrusage` on the same runs. System time grows faster than user time and tracks the resident set and the fault count, which identifies memory acquisition as the cost.

8.11 Device Saturation (Table T11)

The megakernel is launched at a fixed number of blocks per streaming multiprocessor, so that number is a tuning parameter rather than a property of the problem. Throughput improves to

Table 12: **T14 — The deepest point reached by each left-hand side shape.** One row per shape, at the greatest depth that both fitted the collection budget and did not reach a container ceiling (see Figure 9), so the depths differ between rows by construction and are reported rather than held equal. `Shape` is the connectivity, `Arity` the vertices per edge, `LHS` the number of edges in the pattern. The state, causal and branchial columns are the sizes of the three objects the run built. Rows that agree exactly are not duplicates: raising a path's arity past three changes nothing the matcher sees, because consecutive edges already overlap in enough vertices to pin a match's orientation, so `chain2a3` and `chain2a4` describe the same search and report the same counts. Source: `tools/dev/rich_sweep.sh` over `tools/sampling_cost_smoke`.

Rule	Shape	Arity	LHS	Depth	States	Causal	Branchial
`chain1a2`	chain	2	1	6	1,339,281	850,896	0
`growth`	chain	2	1	10	4,037,914	4,037,912	0
`chain2a2`	chain	2	2	5	61,048	92,164	74,609
`pair`	chain	2	2	7	204,556	393,788	181,439
`chain2a3`	chain	3	2	5	19,608	26,732	16,806
`chain2a4`	chain	4	2	5	19,608	26,732	16,806
`chain3a2`	chain	2	3	5	33,649	87,920	81,725
`triple`	chain	2	3	6	23,116	66,980	37,362
`chain3a3`	chain	3	3	5	9,331	23,612	13,995
`chain4a2`	chain	2	4	5	17,046	63,276	55,760
`quad`	chain	2	4	6	23,116	90,092	51,609
`cycle3a2`	cycle	2	3	9	29,524	88,560	29,523
`cycle4a2`	cycle	2	4	7	21,845	87,360	32,766
`disc2a2`	disc	2	2	4	309,853	547,488	3,238,014
`disc`	disc	2	2	4	309,853	547,488	3,238,014
`disc3a2`	disc	2	3	4	1,045,591	3,136,752	58,835,751
`mixed2`	mixed	2	2	8	87,381	148,520	0
`mixed3`	mixed	2	3	9	29,524	85,978	19,682
`mixed4`	mixed	2	4	9	29,524	115,976	19,682
`star2a2`	star	2	2	4	26,293	45,792	108,378
`star3a2`	star	2	3	3	14,425	40,176	165,876
`star4a2`	star	2	4	3	14,425	57,600	165,876
`tree3a2`	tree	2	3	5	34,599	89,552	82,480
`tree4a2`	tree	2	4	5	40,655	148,628	179,283

Table 13: **T13 — User and system time, resident set and page faults against thread count.** A declining efficiency curve does not by itself identify the resource being consumed, and the two candidate explanations—contention on shared structures, and the cost of memory the run acquires—imply different remedies. Both are read from `getrusage` on the same runs. Across a 32× increase in worker count, user time increases by a factor of 4.7—far less than the thread count, so the added workers are not simply spinning against each other—while system time increases by a factor of 9 and tracks the peak resident set and minor-fault count the table gives per row. Each worker draws its own arena blocks, so the run acquires memory in proportion to its width and faults each page in. System time is the faster-growing of the two and the one that scales with the resident set, identifying memory acquisition rather than contention as the cost. Wall time is minimized at sixteen threads and rises again at 32, beyond which this kernel cost exceeds the parallel gain. This result is specific to the host measured; a host with different memory bandwidth or huge pages enabled would place the minimum elsewhere. Source: `tools/dev/scaling_sweep.py` over `tools/sampling_cost_smoke` under `/usr/bin/time -v`.

Threads	Wall s	User s	System s	Peak RSS MB	Minor faults
1	3.09	2.73	0.35	624	160 040
8	1.00	6.36	0.80	1184	311 547
16	0.80	8.46	1.51	1816	483 330
24	0.77	10.60	2.39	2377	633 698
32	0.82	12.73	3.25	2865	768 659

eight blocks per SM and is flat thereafter, which is where the resident frontier stops being the limit. The reported utilization upper bound is 100% at every point, so the flattening is not that bound being reached.

Table 14: **T11 — Device saturation against grid size.** The resident kernel's grid size is a run-time parameter, so saturation is determined behaviorally: wall time decreases while additional blocks contribute work and flattens when they do not. It flattens at eight blocks per streaming multiprocessor—1.87× the throughput of one block per SM, with sixteen per SM within measurement variation of eight—which is the shipped default. The final column is the driver's peak SM-busy reading during the run, sampled at 20 Hz. It is an upper bound on utilisation and not an occupancy figure, since the counter reports whether the SMs were busy rather than how many warps were resident. The mean of that counter is not reported: it decreases as the run becomes faster, because the same busy kernel occupies a smaller fraction of the sampling window, and therefore tracks duration rather than residency. Source: `tools/dev/scaling_sweep.py` over `tools/bench_gpu_evolve` mode 2.

Blocks per SM	Blocks	Median ms	vs. 1 per SM	Peak SM-busy
1	128	206.08	1.00×	100%
2	256	156.41	1.32×	100%
4	512	125.96	1.64×	100%
8	1024	110.94	1.86×	100%
16	2048	111.60	1.85×	100%

8.12 Occupancy, Lane Efficiency, and the Bounds They Set

Table 14 establishes saturation behaviorally and states that its final column is not an occupancy figure. This section supplies the occupancy figure and the second bound that sits beside it, both read from the shipped binary rather than inferred from timings, so that the device results above are reported against the ceilings they were measured under.

Two resources bind the resident kernel, and they multiply rather than compete.

Registers. Reading the linked binary with `cuobjdump -res-usage`, `k_persistent_evolve` is allocated 255 registers per thread, which is the architectural maximum. The kernel is launched with 32 threads per block, so a block reserves $255 \times 32 = 8{,}160$ registers; a streaming multiprocessor on the sm_89 part used here provides 65,536, admitting eight resident blocks, or 256 threads against a hardware maximum of 1,536. Occupancy is therefore 16.7%, and it is register-bound rather than bound by shared memory (48 bytes per block) or by block count. Per-translation-unit object files report no stack or local frame for any function under separate compilation (`-rdc=true`); only the linked binary carries the allocation, and reading the intermediate objects understates it to zero.

Lane efficiency. The rewrite, canonicalization, deduplication and quotient stretch of the kernel—about 113 lines, and the phase the measurements below show to dominate—executes inside a single `if (threadIdx.x == 0)` guard. Thirty-one of the block's thirty-two lanes are idle for its duration, so the achieved SIMD width over that stretch is 1/32, or 3.1%.

Hardware counters exclude every memory and arithmetic alternative, and a direct experiment excludes occupancy itself. No unit is near its limit: on this kernel DRAM throughput is 0.71% of peak, L1 3.28%, L2 5.01% and SM throughput 3.11%. These are counters from a separate profiler session on one workload and are not the corpus figures of Table 15, which are taken at a different depth and configuration; both say the same thing, that no unit is close to saturation. Atomic traffic, which the work queues make the natural suspect, is 157,529 operations in a 1.22 ms kernel and lands in L2 at that 5.01% figure, leaving twenty times' headroom.

Occupancy requires more care than the register figure alone suggests, because two independent limits coincide. The resident kernel is allocated 255 registers, which over a 32-thread block permits eight blocks per streaming multiprocessor; the kernel is also launched with a grid of eight blocks per multiprocessor. Both give eight warps of a possible forty-eight, or 16.7%, and a measured 16.95% of peak warp slots active is consistent with either. They are separable by experiment. Capping the whole device library at 128 registers halves the allocation and leaves occupancy unchanged at 16.66%, so the register ceiling was never the binding constraint. Raising the grid with that cap in place does raise residency—to 33.30% at sixteen blocks per multiprocessor—and the kernel becomes *slower*, 1.23 ms to 1.31 ms, degrading further to 1.43 ms as the grid grows. This is consistent with the behavioral saturation of Table 14 and identifies its cause: additional resident warps contend for the shared work cursors without contributing independent work.

The device is therefore limited neither by bandwidth, nor by queue contention, nor by occupancy. It is limited by the dependency structure of the work itself. Individualization–refinement is a depth-first backtracking search in which each step depends on its predecessor, and no quantity of resident parallelism executes a dependent chain faster. Of the refinement's instruction count, 40% lies in scans over vertices and edges that could be distributed across a warp and 60% in the search itself; distributing the former perfectly bounds the phase at 1.63×, and at 51–77% of block cycles that is 1.25–1.42× overall. That is the ceiling for this algorithm on this architecture, and it is a property of canonicalization rather than of the implementation.

Taken as a product, under 1% of the device's arithmetic capability is reachable by the dominant phase. The measured throughput reflects that bound, and it is a structural property of the current work decomposition rather than a scheduling artifact or a stall: an independent instrument (below) reports zero time spent waiting on unpublished work.

Per-phase attribution, taken from `clock64` deltas accumulated per block and summed at exit, locates the cost. On the two corpus workloads that generate enough work at depth six to attribute anything—`wolfram24` at 9,820 states and `wpp` at 3,867—canonicalization accounts for 76.7% and 51.0% of block cycles respectively, against 2.7% for rewriting and 0.3% for matching in both. The remaining corpus shapes produce between four and 104 states and are 97.4% to 99.9% idle; that is the absence of work rather than a property of the scheduler, and their idle fraction should not be read as a device inefficiency. Canonicalization is thus both the dominant

phase and resident in the single-lane region, and the two bounds above identify the same site in the code by independent routes.

A third bound sits outside the kernel, in the per-call bracket rather than in the evolution itself. Tracing a device run and placing every allocation and copy relative to the resident kernel's own execution: no copy, allocation or memset occurs while the kernel is in flight—the count is zero across every evolution measured—so the evolution proper is free of host traffic, as the design requires. The bracket around it is not. Each evolution runs a 3.8 ms kernel preceded by 189 and followed by 189 runtime allocation and copy calls (23 allocations, 23 frees, 50 copies and 93 memsets on each side). The design goal of a single readback at the end is therefore not met, though the stronger property of no traffic during evolution is. This bounds interactive and session use, where a caller pays that bracket on every call, rather than throughput on a single deep run, where it is amortized over the kernel's duration.

Every device figure above is measured with the record set off, and that configuration hides a second regime. The reconstruction of the raw causal and branchial relations is selectable independently of the evolution, and the measurements reported so far switch all of it off. Re-measured with causal recording on, the device and host agree *exactly* on the reconstructed relation for the eight corpus workloads the device backend supports, at depth six—identical distinct-pair counts and identical counts surviving transitive reduction. The two implementations share no code below the shared replay core, so the agreement is between two separate schedulers, memory layouts and queueing disciplines. On the densest workload available, a three-edge disconnected left side at depth three, the two engines agree on 1,661,760 causal pairs and 970,560 surviving reduction.

Reaching that agreement is a property of how the reduction is defined rather than of how it is computed. A finite directed acyclic graph has exactly one transitive reduction, so membership is a function of the finished relation; both engines therefore store the relation and reduce it on demand, rather than deciding each pair as it arrives against the pairs seen so far. The two engines share that body. Deciding on arrival makes the answer depend on the schedule: a pair reaching the relation before the longer path that bypasses it is kept, the same pair reaching it afterwards is dropped, and an append-only adjacency never retracts the difference. The reduction is computed once per source rather than once per pair—every edge leaving a given event shares the set reachable from it in two or more steps—and where event identifiers increase along every causal edge the search is bounded by the largest target of that source. The branchial relation is handled the same way: a branchial pair is two applications of one instance sharing a consumed slot, so neither engine stores the pairs, and both derive them from the applications on demand.

The idle fraction of the low-work shapes is not, in general, the absence of work. The attribution above reports that the corpus shapes producing few states are 97.4–99.9% idle and reads that as work not existing. With causal recording on this reading no longer holds for two of them: at depth six `cycle4` carries 95,600 causal pairs and `multirule` 177,201, and both remain above 98.7% idle while taking 69 and 104 ms. Hardware counters place them far from every ceiling—SM throughput 1.55% of peak and DRAM 0.01% on `cycle4`—and its lane efficiency is the corpus's *best* at 11.37 of 32, against 1.29 on `wolfram24`, which the device completes 3.4× faster than the host. The decisive comparison is that the kernel is slower on 49 states (104 ms) than on 9,820 (9.7 ms).

A depth sweep identifies what binds, and rules out the reading that first suggests itself. Holding the workload fixed and varying depth, `cycle4` produces 152, 1,232, 10,400 and 95,600 causal pairs at depths three to six, over 16, 23, 31 and 40 states, in 3.5, 5.0, 12.4 and 70 ms. Were this a serial chain whose length is the reconstruction depth, the time would follow the depth: it would double from depth three to depth six, and it grows twentyfold. Nor is it linear

in the work: the pair count grows 629× across that range while the time grows 20×, so the cost per pair *falls* from 23 to 0.73 µs, a factor of 31. Parallel efficiency improves with scale rather than saturating, which is the opposite of a fixed serial bottleneck.

What separates the two regimes is the parallel width, and the corpus supplies a controlled comparison for it. At depth six `cycle4` carries 95,600 pairs over 40 states and loses to the host; `wolfram24` carries a comparable 130,476 pairs over 9,820 states and beats it by 3.4×. The pair counts are within 40% of each other and the outcomes are opposite, so neither the size of the relation nor the depth of the recursion is the discriminator. The state count is: a class is the unit over which the replay's instances are independent, and a run that collapses to forty classes has forty-fold width available however much work those classes carry. `cycle4` begins from a deliberately automorphic initial state, which is exactly the condition that collapses classes. The device's disadvantage on these shapes is therefore a shortage of independent work rather than an inability to execute the work it has, and raising occupancy against it would raise the idle count rather than the throughput.

The instances *within* a class can be replayed concurrently. They are serialized by an implementation choice rather than by the reconstruction: each driver takes its instances from a worklist private to it, so a class is walked by one driver and, on a run that collapses to forty classes, forty such walks proceed while the remainder of the grid spins.

Nothing in the reconstruction requires this. A child instance is published to the instance index behind a fence before it is driven, and each (instance, match) application is claimed through a device-wide set, so an application is idempotent and safe from any thread. A push visible across drivers would therefore produce the same result as one confined to the driver that discovered the instance, and the available width would be the instance count rather than the class count.

The two are far apart on exactly the shapes that lose. At depth six `cycle4` materialises 68,185 instances over 40 classes and `multirule` 146,599 over 49—deficits of 1,705× and 2,992×—while `wolfram24` materialises 86,086 over 9,820 and `wpp` 15,967 over 3,867, deficits of 8.8× and 4.1×. The instance counts are comparable across all four, between 16 and 147 thousand, while the class counts vary by a factor of 250: the independent work available is similar everywhere and only the width the implementation exploits differs. The shapes that lose are those whose initial state collapses classes, and that collapse is the same condition that concentrates instances into few of them.

The residual on these two workloads is accordingly attributed to a named and evidenced cause that is *reducible*: a depth-first descent performed inline by the discovering thread, in a design whose own publication and claim machinery already admits the alternative. It is not claimed as a lower bound, and the corresponding optimisation is identified and unimplemented rather than refuted.

Bounds these results were measured against. Nothing in this corpus is near a hardware ceiling with the record set on: across `wolfram24`, `wpp` and `cycle4` the highest DRAM figure is 6.82% of peak and the highest SM-throughput figure 11.53%. Occupancy is 16.67% on all three to four significant figures, structural for the reason given above. Lane efficiency separates the two regimes: 4.0% and 6.7% on the workloads the device wins, where divergence binds, and 35.5% on the one it loses, where latency on a serial chain binds. The allocation invariant holds in this configuration as well—profiling twenty-four consecutive evolutions with the reconstruction running records zero allocations, copies or memsets between a resident kernel's start and its end, with all 5,042 such calls falling in set-up or read-back. What is not established is whether the serial chain is irreducible; it is attributed and bounded, not proven minimal.

The quotient decoupling of Section 5.10 applies to the device on the same terms as the host. Capture of each class's frame is separated from replay of that capture against instances. The first signs event identity at the cost of the canonical answer, the second recovers the raw event

set at the cost of the raw one. Together they leave the canonical answer bit-identical while removing the exponential from runs that do not request raw events. Two corpus shapes do not complete depth seven under unconditional replay at all. On `multirule` the resident kernel's median falls from 183.4 ms to 4.14 ms at depth six, and from a timeout beyond 150 s to 4.59 ms at depth seven; on `cycle4`, from 107.6 ms to 5.27 ms and from a timeout to 5.27 ms. State and event counts are identical in both arms. Depth six to seven then costs 4.14 → 4.59 ms and 5.27 → 5.27 ms: flat, where the unconditional arm does not terminate.

Two routes to the canonicalization bound were measured and rejected. Both are reported here because the bound remains open, and because the two fail for opposite reasons. Tiered canonicalization—bucket by the Weisfeiler–Leman signature, run individualization–refinement only on a collision—was implemented and measured at 28% slower, because a bucket hit still requires refinement to confirm isomorphism, so a duplicate state is processed by both stages. A content-equality short-circuit reuses a canonical form when two raw states hold literally the same edge tuples. It needs no confirmation, being a proof of isomorphism on its own—and it never fires. Measured over `wpp`, the number of distinct state contents equals the number of raw states exactly at depths six and seven (3,867 of 3,867; 45,317 of 45,317), because every rewrite mints fresh vertex identifiers, so no two states reaching one canonical form carry the same labels. The measurement is order-independent and therefore an upper bound on what any content key could catch. The two routes fail oppositely: the first fires often but confirms redundantly, the second would need no confirmation but never fires. The 1.59× gap between refinement passes performed and distinct canonical states is real; reaching it requires recognising isomorphic states without canonicalizing them, which is the problem canonicalization solves.

These figures hold across the corpus rather than on one shape. Profiling the resident kernel on all eight workloads at depth six—measured with the profiler rather than by wall clock, so that occupancy and throughput are read from hardware counters rather than inferred from elapsed time—gives occupancy between 16.58% and 16.66% on every one, over workloads whose kernel durations differ by a factor of seven hundred (Table 15):

Table 15: **Resident-kernel occupancy and throughput across the corpus.** Read from hardware counters under NVIDIA Nsight Compute at depth six, not inferred from elapsed time. Occupancy is between 16.58% and 16.66% on every workload, over kernel durations differing by a factor of seven hundred as measured, though two of those durations are profiler artifacts and the real spread is nearer a factor of two (see below); the invariant holds across either reading, which is what makes it structural rather than workload-dependent. Unlike every other table in this section this one is transcribed from a profiler session rather than emitted by `tools/dev/paper_tables.py`: Nsight Compute reports are not text, and the session is recorded as `ncu_wpp.ncu-rep` alongside the other artefacts of the run.

workload	kernel (ms)	warps active	SM thr.	DRAM thr.
`wpp`	2.97	16.66%	9.37%	7.51%
`wolfram24`	5.81	16.66%	13.00%	9.49%
`multirule`	1.23	16.66%	2.06%	0.06%
`cycle4`	1.72	16.66%	1.79%	0.05%
`multiroot`	3.29	16.66%	2.00%	0.04%
`arity3`	3.55	16.66%	1.62%	0.01%
`triangle`	209.38	16.58%	2.04%	0.21%
`binary`	857.47	16.65%	1.80%	0.06%

A grid-determined limit produces exactly this: an occupancy constant across a corpus this varied, and confirms from a third direction that it is not determined by register allocation.

Two entries in that table are artifacts of the profiler rather than properties of the workloads.

The profiler serializes execution and fixes clocks, so a kernel whose resident blocks sit in a backoff loop rather than computing has its duration inflated by one to two orders of magnitude. Measured unprofiled, `binary` completes in 3.87 ms and `triangle` in 3.39 ms against the 857 ms and 209 ms above, while `wpp`, which is genuinely computing throughout, takes 7.52 ms unprofiled against the 2.97 ms the profiler reports—lower there because the profiler excludes the host-side bracket around the launch. The workloads that generate little work are therefore faster than the ones that generate much, as expected, and their 99.5% and 99.7% idle phase readings are 99.5% of four milliseconds rather than of a second.

The occupancy and throughput columns are unaffected. Those are ratios and structural properties rather than elapsed times, and the table reports them for that reason: an occupancy invariant at 16.6% across workloads whose real durations span a factor of two, and utilisation figures that never approach any unit's limit.

We report these bounds rather than a closed gap. The decomposition that would raise lane efficiency—distributing canonicalization across the warp—and the compile-time separation that would relax the register ceiling are identified and unimplemented at the time of writing. Raising occupancy alone would multiply a 3.1% lane efficiency by a larger factor and report a speedup while the device continued to run one lane in thirty-two, so the ordering matters and the ceiling is stated here as measured rather than as addressed.

8.13 Ablation (Table T9)

Only two of the engine's contributions remain run-time switches; the rest were replaced outright and their predecessors deleted in the same commit, so no "off" arm exists to measure. Match forwarding is the interesting case: it is worth 1.47× on a two-edge left-hand side and *costs* 1.26× on a one-edge one, so it is decided per rule set by the static analysis of Section 5.6 rather than being switched on globally. Both arms of every row compute the same state and event counts.

The canonicalization row is the one that cuts against the framing of Section 3. Exact individualization–refinement is not the expensive option here: it runs at 195.3 ms against 211.3 ms for the cheaper content hash, a ratio of 0.92. The hash does not remove the refinement pass, it adds a second key on top of it—the sampling draw is keyed on `canonical_transition_key`, whose first act is the same refinement—so on this workload the approximate mode pays for both. That is the same result Proposition 3.3 predicts from the other direction: a cheaper filter in front of IR cannot remove the IR invocations, so it can only add to them.

8.14 Where the Instructions Go (Table T15)

Wall time says how long a run took; it does not say which part of the engine spent it, and on a machine whose clock drifts between sessions it is the wrong instrument for attribution. Executed instruction counts are deterministic, so this table is taken once under `callgrind` on a single-threaded depth-six run and needs neither repeats nor a quiet machine. Matching, not canonicalization, is the largest component; canonicalization is second. The unattributed share is the standard library, the runtime and functions below the reporting threshold, and it is reported rather than distributed over the named buckets.

8.15 Event Canonicalization Overhead

Event identity is not one convention but three, and they disagree in general rather than in corner cases. Table 18 evolves each workload once and counts the distinct events under each: identifying an event by its endpoint states, by the edges it consumed and produced, and by the distinct rule applications. On `binary-growth` the three give 5, 8 and 9. A paper reporting an event count must therefore say which convention it used, and this one uses the canonical convention throughout.

Table 16: **T9 — Per-contribution ablation over the run-time switches present in the shipped binary.** No compiled-out ablation builds exist: when a replacement lands in this engine the replaced path is deleted in the same commit, so an "off" arm exists only where the choice remains a run-time switch. Those are match forwarding, whose applicability depends on the rule set (Section 4), and the state-canonicalization mode; quotient exploration is reported separately in Table 7. Each arm is a best-of-N wall time. The forwarding rows use a one-edge left-hand side at depth eight and a two-edge left-hand side at depth six; the canonicalization row uses the two-edge side at depth six. State and event counts are compared across arms, so a switch that alters the computed result is identified as such rather than reported as a change in speed—but that column is a statement about *these* workloads. The state-canonicalization mode does change the computed answer on other inputs: Section 5.14 gives a corpus case where the exact and approximate modes report eight events and six. The row below reports agreement because the two-edge left-hand side at depth six is not such a case, which is exactly why the workload is named here rather than left implicit. Source: `tools/sampling_cost_smoke`.

Contribution	Arm A	A ms	Arm B	B ms	Ratio	Same answer
Match forwarding, one-edge LHS	on	163.2	decided off	140.2	1.16	yes
Match forwarding, two-edge LHS	off	274.3	decided on	177.7	1.54	yes
State canonicalization	Full (exact)	178.3	Automatic (hash)	195.3	0.91	yes

Table 17: **T15 — Executed instructions by component.** A single-threaded depth-six evolution of the Wolfram rule under `callgrind`, grouped by the file each function is defined in. Instruction counts are exact and reproducible, so attribution uses them rather than wall time. The flat per-function profile covers 99.0% of the program total; the remainder is below `callgrind_annotate`'s reporting threshold. Source: `tools/dev/instruction_profile.py` over the `attrib` phase of `tools/dev/remote_session.sh`.

Component	Instructions	Share
Match expansion and join	117,653,606	24.7%
Canonicalization (IR)	79,592,324	16.7%
Dedup sets and maps	71,807,564	15.1%
Arena allocation	38,097,485	8.0%
Hypergraph storage	24,250,782	5.1%
Causal and branchial	728,120	0.2%
Job system and work stealing	366,201	0.1%
Unattributed	139,143,506	29.2%
Total	476,371,312	99.0%

Table 18: **Event identity is three distinct conventions, not one under three names.** Canonical event count per convention on each corpus workload, under FULL state canonicalization, single-threaded. The conventions refine one another — two rewrites are the same event when they run between the same states, when they additionally move the same edges, or never — so the counts must be non-decreasing left to right, and where they are strictly increasing the convention is doing work no coarser one does. There is no timing column: what separates the conventions is what they IDENTIFY, and the cost of the identity is the added canonicalization pass that Table 16 measures. Source: `tools/mode_matrix_probe.cpp`.

Workload	Steps	Canon. states	By endpoint states	By consumed/produced edges	Distinct applications
binary-growth	3	6	5	8	9
wolfram-2to4	2	4	3	4	6
triangle	2	3	2	4	4
reductive-2to1	3	4	3	6	15
idempotent-2to2	3	1	1	1	3
self-loop	3	2	1	1	1
mixed-arity	3	4	3	3	3
arity3-growth	3	9	9	9	9
disconnected-lhs	3	1	1	4	14
arity4-growth	3	9	9	9	9
mixed-arity-lhs	3	2	1	1	1
mixed-arity-init	3	4	4	4	4
arity3-to-2	3	4	4	4	4
arity1-with-binary	3	2	1	1	1
cycle4-automorphic	3	4	3	15	144
star4-automorphic	2	5	4	12	24
multi-rule	2	10	11	11	11

8.16 Limitations

Single machine. All numbers were taken on one desktop, whose configuration is given in Section 8 and repeated in each table's provenance line. The scaling results in particular are properties of this machine's memory system and core count, and the thread count at which efficiency declines is not portable to a machine with a different one.

Quotient reconstruction scales with the system it avoids visiting. Quotient exploration visits one representative per isomorphism class, and its state count can grow far more slowly than the raw system's. The reconstruction that recovers raw observables from those representatives does not share that saving: it materialises instances, and there are as many instances as the raw system holds. Measured on a two-rule set whose canonical state count grows linearly with depth, the reconstruction's cost grew about twentyfold per step—0.17, 0.22, 0.62, 7.36, 144.78 and 4981.05 milliseconds at depths two through seven for three, four, five, six, seven and eight canonical states. The growth rate is a property of the output rather than of the implementation: the raw observables are as numerous as the raw system holds, whatever computes them. The cost applies only to runs that request those observables. The reconstruction is enabled by the request, so a run asking for canonical states alone does not perform it.

The reconstruction does not scale with threads. Every worker drives it, and the total cycles it consumes grow with the thread count—measured at 511 million cycles on one thread against 3.85 billion on thirty-two for the same workload, with the same amount of admitted work. The dedup marks prevent duplicate effects but not duplicate traversal, so added threads contend rather than divide the work. On that workload wall time improves to eight threads and degrades beyond it.

Two comparison points, one of them our own. The implementations compared against are the `Wolfram/Multicomputation` paclet's `MultiwaySystem` and this project's own reference implementation. The first is the only external implementation known to us that computes the deduplicated states graph under an isomorphism identity; SetReplace computes a different object

(Section 2), so no wall-time comparison against it is reported. The second is written by the same author as the engine, from the same definition, which makes it an independent implementation of the definition but not an independent party's implementation.

Workload scope. Two corpora are used and they are different sizes. The oracle corpus of `reference/oracle_corpus.hpp` is 17 rule-type workloads and is what Tables 4, 5 and 18 report; the left-hand-side sweep of Section 8.8 generates 24 shapes of its own, reported by Table 12. Both were constructed to cover rule shapes—productive, reductive, idempotent, mixed arity, disconnected and high-automorphism—rather than to sample rules found in practice, and the observed behavior of C/R and of parallel efficiency is a property of those shapes.

Scope of the concurrency results. The model-checked properties hold for the thread and operation counts given, which are small; they are not proofs for unbounded thread counts. The determinism gate likewise establishes agreement across the configurations it runs, not for all.

Worst-case cost. Individualization–refinement is exponential in the worst case, and the exact mode applies it to every state. States with large automorphism groups are where that cost is realized, and Proposition 3.3 shows that a cheaper pre-filter cannot avoid it.

9 Conclusion and Future Work

We have presented an implementation of multiway hypergraph evolution built on exact canonicalization and lock-free concurrency. Compared with the `Wolfram/Multicomputation` paclet's `MultiwaySystem` on the same rule and initial condition, and producing identical state and causal-edge counts at every depth, the engine is faster by a factor of 2,770 at depth 4, rising to 30,627 at depth 7 (Table 6). The ratio rises monotonically across those four depths. At the shallow end of the range the engine's own time is 0.2 to 0.5 ms, close enough to the timer's resolution that those ratios are better read as lower bounds on the gap than as exact factors; at depth seven it is 23.2 ms against 710 s, so that row is a direct measurement on both sides. The same table compares the engine against `MultiwayReference`, this project's own brute-force evolver, which computes the same object from the same definition and returns data rather than graph objects, so that ratio is not attributable to graph construction on the slower side; all three agree on state and causal-edge counts at every depth. On the host, evolution attains a speedup of 7.5× at 16 worker threads, with parallel efficiency higher on the larger problem at every thread count beyond the first (Table 10). Efficiency remains at or above 0.43 up to eight workers and at or above 0.31 up to sixteen, depending on the rule's shape, and declines beyond that (Table 11). The measured cause is memory acquisition rather than contention: across a 32× increase in worker count, user time increases by a factor of 4.7 while system time increases by a factor of 9 and tracks the per-worker resident set and minor-fault count (Table 13). On the device, wall time ceases to improve beyond 8 blocks per streaming multiprocessor (Table 14), and the GPU is 6.2× faster than eight CPU threads at depth 7 (Table 8). The contributions are:

1. Exact canonicalization via individualization–refinement (IR) computed on every state, its hash serving directly as the deduplication key, validated against a brute-force isomorphism oracle

2. Lock-free concurrent data structures, whose properties are model-checked rather than stress-tested (Section 7)

3. A resident CUDA kernel that carries matching, rewriting and canonicalization without returning to the host between evolution steps

4. Independent, exact state and event equivalencing axes with configurable granularity

5. Quotient exploration, transition-level sampling, and an online transitive reduction of the causal graph, each preserving the observable contract of Section 5.8

6. A determinism contract: the state set, event set and causal and branchial relations are a function of the inputs alone, independent of thread count and scheduling order (Section 5.8)

9.1 Future Directions

Four directions remain open. The first is performance and is named by a measurement in this paper rather than by expectation; the other three are scope.

Instance-parallel replay: The reconstruction drives each instance from a worklist rather than by recursion, but the worklist is private to one driver, so the width it exploits is the class count rather than the instance count. On the class-collapsed workloads the two differ by four orders of magnitude: `pair` on a 24-edge path expands 373,176 states that fall into 5 canonical classes. Making the worklist visible across drivers is what would spend the instance count instead, and the publication and claim machinery a shared push requires is already present.

Distributed computation: The current design targets single-node execution. Extending to distributed memory systems would enable exploration of larger state spaces than one machine's memory admits.

Rule learning: Automatic discovery of rules producing desired structures (e.g., emergent dimensionality, causal invariants) remains an open challenge amenable to learned heuristics.

Verification beyond the bound: The properties of Section 7 are established for bounded programs under RC11 and, for the forwarding protocol, over an unbounded worker count against a transcribed model. Neither establishes the code's properties for unbounded thread counts. A mechanised proof over the implementation itself would.

9.2 Availability

The engine is used directly as a library and as the evolution backend for downstream tools; with `BUILD_VISUALIZATION` enabled it emits an event stream that an external renderer consumes to draw the evolution, causal and branchial graphs. The implementation is available as:

- Source code: https://github.com/WolframInstitute/HypergraphRewritingEngine
- Wolfram Language paclet: `WolframInstitute/HypergraphRewriteEngine`, installable with `PacletInstall["WolframInstitute/HypergraphRewriteEngine"]`

Acknowledgments

The author thanks Stephen Wolfram for foundational work on the Wolfram Physics Project. This work was supported by the Wolfram Institute.

References

[1] Stephen Wolfram. A project to find the fundamental theory of physics. https://www.wolframphysics.org, 2020. Wolfram Media.

[2] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014.

[3] Boris Weisfeiler and Andrei Lehman. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsia*, 2(9):12–16, 1968.

[4] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. Monographs in Theoretical Computer Science. Springer, 2006.

[5] Grzegorz Rozenberg, editor. *Handbook of Graph Grammars and Computing by Graph Transformation*, volume 1. World Scientific, 1997.

[6] Detlef Plump. The graph programming language GP. In *Algebraic Informatics*, pages 99–122. Springer, 2009.

[7] Johannes Köbler, Uwe Schöning, and Jacobo Torán. *The Graph Isomorphism Problem: Its Structural Complexity*. Birkhäuser, 2012.

[8] László Babai. Graph isomorphism in quasipolynomial time. *Proceedings of the 48th Annual ACM SIGACT Symposium on Theory of Computing*, pages 684–697, 2016.

[9] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-Lehman graph kernels. *Journal of Machine Learning Research*, 12:2539–2561, 2011.

[10] Wolfram Physics Project. SetReplace: A Wolfram language package and C++ library for hypergraph rewriting. https://github.com/maxitg/SetReplace, 2020.

[11] Wolfram Research. Wolfram/multicomputation. Wolfram Language Paclet, 2023. URL https://resources.wolframcloud.com/PacletRepository/resources/Wolfram/Multicomputation/. Provides `MultiwaySystem`, the authority this work is compared against.

[12] Zhengyi Yang, Wenjie Zhang, Xuemin Lin, Ying Zhang, and Shunyang Li. HGMatch: A match-by-hyperedge approach for subgraph matching on hypergraphs. In *IEEE International Conference on Data Engineering (ICDE)*, pages 2063–2076, 2023.

[13] Ha-Nguyen Tran, Jung-jae Kim, and Bingsheng He. Fast subgraph matching on large graphs using graphics processors. In *Database Systems for Advanced Applications*, pages 299–315. Springer, 2015.

[14] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proceedings of the VLDB Endowment*, 13(8):1190–1205, 2020.

[15] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. *ACM SIGPLAN Notices*, 51(8):1–12, 2016.

[16] Xuhao Chen and Arvind. Efficient and scalable graph pattern mining on GPUs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[17] Gramoz Goranci, Adam Karczmarz, Ali Momeni, and Nikos Parotsidis. Fully dynamic algorithms for transitive reduction. In *International Colloquium on Automata, Languages and Programming (ICALP)*, volume 334 of *LIPIcs*, pages 92:1–92:20, 2025.

[18] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.

[19] Jaco van de Pol and Michael Weber. The fastest term rewrite engine. *Electronic Notes in Theoretical Computer Science*, 343:169–187, 2019.

[20] Gian-Carlo Rota. The number of partitions of a set. *The American Mathematical Monthly*, 71(5):498–504, 1964.

[21] Albert Atserias, Martin Grohe, and Dániel Marx. Size bounds and query plans for relational joins. *SIAM Journal on Computing*, 42(4):1737–1767, 2013.

[22] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. Worst-case optimal join algorithms. *Journal of the ACM*, 65(3):16:1–16:40, 2018.

[23] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2012.

[24] Maged M. Michael. High performance dynamic lock-free hash tables and list-based sets. In *Proceedings of the Fourteenth Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 73–82, 2002.

[25] Timothy L. Harris. A pragmatic implementation of non-blocking linked-lists. In *International Symposium on Distributed Computing (DISC)*, pages 300–314. Springer, 2001.

[26] David Chase and Yossi Lev. Dynamic circular work-stealing deque. In *ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 21–28, 2005.

[27] Nhat Minh Lê, Antoniu Pop, Albert Cohen, and Francesco Zappa Nardelli. Correct and efficient work-stealing for weak memory models. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 69–80. ACM, 2013.

[28] Michalis Kokologiannakis and Viktor Vafeiadis. GenMC: A model checker for weak memory models. In *Computer Aided Verification (CAV)*, volume 12759 of *Lecture Notes in Computer Science*, pages 427–440. Springer, 2021.

[29] Ori Lahav, Viktor Vafeiadis, Jeehoon Kang, Chung-Kil Hur, and Derek Dreyer. Repairing sequential consistency in C/C++11. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 618–632. ACM, 2017.

[30] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

A Complexity Analysis Details

This appendix gives the bounds quoted in the body, for the two operations that dominate a run: deciding whether a produced state has been seen before, and finding the matches of a rule in a state. Both have a worst case far above their observed cost, and in both the gap is a property of the *input* rather than of the implementation—symmetry in the first case, left-hand side connectivity in the second—so each bound is stated together with what makes it attainable. Appendix B gives the per-operation costs of the concurrent structures these bounds are realized on (Table 19).

A.1 Canonicalization Complexity

For a hypergraph with $|V|$ vertices and $|E|$ hyperedges of maximum arity k, the two canonicalization roles have different cost profiles.

Weisfeiler–Leman signature. One refinement round aggregates, for each vertex, the multiset of neighbor colors: $\mathcal{O}(k \cdot |E|)$ work per round, with at most $|V|$ rounds before the coloring stabilizes, giving $\mathcal{O}(|V| \cdot k \cdot |E|)$ in the worst case and far less in practice (the partition typically stabilizes in a few rounds). WL is a sound but incomplete invariant: identical hashes are necessary but not sufficient for isomorphism.

Individualization–refinement (exact). IR interleaves color refinement with individualization of vertices in non-singleton cells, exploring a search tree pruned by discovered

automorphisms. It is polynomial on instances that refinement alone resolves, but—like nauty and Traces—exponential in the worst case, with the blow-up concentrated on highly symmetric (high-automorphism) states. We claim no polynomial-time bound. Under the exact mode IR is applied to every state, and profiling attributes 16.7% of the engine's executed instructions to it (Table 17). Bucketing by a cheaper hash and invoking IR only on collisions does not reduce this cost, by Proposition 3.3: a repeated bucket is consistent with a genuine duplicate, which IR must still confirm, so such a filter elides at most a fraction C/R of the invocations it precedes.

A.2 Pattern Matching Complexity

For pattern with $|L|$ edges and state with $|S|$ edges:

- Signature index lookup: $\mathcal{O}(1)$ per pattern edge
- Candidate enumeration: $\mathcal{O}(|S|/B_d)$ expected where B_d is the Bell number
- Match verification: $\mathcal{O}(|L|)$ per candidate
- Total: $\mathcal{O}(|L| \cdot |S|/B_d \cdot |L|) = \mathcal{O}(|L|^2 \cdot |S|/B_d)$

B Data Structure Specifications

Table 19: **Per-operation costs of the concurrent data structures.** Amortized where the structure grows; n is the number of elements held.

Structure	Insert	Lookup	Delete	Memory
ConcurrentMap	$\mathcal{O}(1)$ amort.	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(n)$
SegmentedArray	$\mathcal{O}(1)$	$\mathcal{O}(1)$	N/A	$\mathcal{O}(n)$
LockFreeList	$\mathcal{O}(1)$	$\mathcal{O}(n)$	N/A	$\mathcal{O}(n)$
SparseBitset	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n/64)$